

國立陽明交通大學
電機資訊國際學位學程
碩士論文

EECS International Graduate Program
National Yang Ming Chiao Tung University
Master Thesis

基於大型語言模型代理之輸入注入攻擊偵測與防禦：
三層過濾架構

Detecting and Preventing Input Injection in LLM-based
Agents with 3-Layer Filtering

研 究 生：黎明朋（Le Minh Bang）

指導教授：林盈達（Ying-Dar Lin）

中華民國一一五年六月

June 2026

基於大型語言模型代理之輸入注入攻擊偵測與防禦：
三層過濾架構

Detecting and Preventing Input Injection in LLM-based
Agents with 3-Layer Filtering

研 究 生：黎明朋

Student: Le Minh Bang

指導教授：林盈達 博士

Advisor: Dr. Ying-Dar Lin

國立陽明交通大學

電機資訊國際學位學程

碩士論文

A Thesis

Submitted to EECS International Graduate Program
College of Electrical Engineering and Computer Science
National Yang Ming Chiao Tung University
in Partial Fulfilment of the Requirements
for the Degree of
Master in
Electrical Engineering and Computer Science

June 2026

Taiwan, Republic of China

中華民國 一一四年六月

基於大型語言模型代理之輸入注入攻擊偵測與防禦： 三層過濾架構

學生：黎明朋

指導教授：林盈達 博士

國立陽明交通大學 電機資訊國際學位學程

摘 要

大型語言模型（Large Language Models, LLMs）已發展為具備推理、外部知識擷取與工具呼叫能力的自主代理，顯著擴展其在真實世界中的應用。然而，這樣的演進也帶來一項關鍵的安全挑戰：輸入注入（input injection）攻擊，即攻擊者可將對抗性指令嵌入多個輸入通道中，進而操控代理的推理過程。不同於傳統的直接提示注入（direct prompt injection），現代攻擊會利用提示、檢索結果與工具輸出等多通道互動，形成關聯式攻擊；當各輸入被獨立分析時，這類攻擊往往難以被偵測。現有防禦方法，包括輸入層級過濾與執行層級驗證，受限於其對單一輸入分析的依賴、對未見攻擊的泛化能力不足，以及較高的計算開銷。為了解決這些限制，本論文提出一種完整互動層級（full interaction-level）的防禦範式，將偵測方式從孤立輸入分析轉向互動層級稽核。本文進一步引入一種級聯式三層過濾架構，包括：(i) 以良性互動軌跡訓練的非監督式異常偵測層，用於捕捉零日攻擊；(ii) 用於偵測已知攻擊模式並降低誤判率的監督式 Transformer 分類器；以及 (iii) 基於大型語言模型的語意稽核器，用於對完整互動歷史進行推理層級驗證。基於 LangGraph 代理測試平台的實驗結果顯示，所提出的方法在維持低誤判率（FPR = 0.0062）的同時，達到平衡的偵測效能（Precision = 0.9932, Recall = 0.9256, F1 = 0.9582），並將所評估之多通道攻擊的攻擊成功率降低至 0%，同時透過級聯式過濾將高成本的大型語言模型稽核限制於尚未解決的案例。這些結果顯示，互動層級防禦能有效且高效率地提升 LLM-based Agent 的安全性。

關鍵詞：大型語言模型代理、輸入注入、三層過濾、異常偵測、轉換器、大型語言模型

Detecting and Preventing Input Injection in LLM-based Agents with 3-Layer Filtering

Student: Le Minh Bang

Advisor: Dr. Ying-Dar Lin

EECS International Graduate Program
National Yang Ming Chiao Tung University

Abstract

Large Language Models (LLMs) have evolved into autonomous agents capable of reasoning, retrieving external knowledge, and invoking tools, significantly expanding their real-world applicability. However, this evolution introduces a critical security challenge: input injection attacks, where adversarial instructions embedded across multiple input channels manipulate the agent's reasoning process. Unlike traditional direct prompt injection, modern attacks exploit multi-channel interactions such as prompts, retrieval results, and tool outputs, forming correlated attacks that remain undetectable when inputs are analyzed in isolation. Existing defenses, including input-level filtering and execution-level validation, are limited by their reliance on single-input analysis, inability to generalize to unseen attacks, and high computational overhead. To address these limitations, this thesis proposes a full interaction-level defense paradigm that shifts detection from isolated inputs to interaction-level auditing. A cascaded three-layer filtering architecture is introduced, consisting of (i) an unsupervised anomaly detection layer trained on benign interaction traces to capture zero-day attacks, (ii) a supervised transformer-based classifier to detect known attack patterns and reduce false positives, and (iii) an LLM-based semantic auditor that performs reasoning-level verification over the full interaction history. Experimental evaluation on a LangGraph-based agent testbed demonstrates that the proposed approach achieves balanced detection performance (Precision = 0.9932, Recall = 0.9256, F1 = 0.9582) while maintaining a low false positive rate (FPR = 0.0062), reduces the attack success rate of evaluated multi-channel attacks to 0%, and limits expensive LLM auditing to unresolved cases through cascaded filtering. These results highlight the effectiveness and efficiency of interaction-level defense for securing LLM-based agents.

Keywords: LLM-based Agent, Input Injection, 3-Layer Filtering, Anomaly Detection, Transformer, Large Language Models

Acknowledgement

回顧這段人生旅程，我滿懷感激之情。這篇論文的完成從來不是一條輕鬆的道路，途中充滿了疑惑、不眠之夜，以及幾度幾乎想要放棄的時刻。然而，在這些困難的時刻推動我繼續前行的，是他人的支持與善意。

首先，我要向我的指導教授林盈達老師表達我最深的感謝。您對「為什麼」的不斷追問，不僅塑造了我研究的邏輯，更深刻地影響了我面對問題的思考方式。您教會我如何更精確、更有耐心地探索問題，並保有好奇心。這些學習將超越論文本身，伴隨我未來的道路。我也誠摯感謝賴源正老師與黃仁竑老師，您們在定期討論中所提供的指導與建議，讓我不斷改進研究內容，並體會到學術清晰與嚴謹的重要性。我特別感謝我的中文老師林佳慧教授，您不僅是出色的教師，更是我真摯的朋友與個人靈感的來源。您溫暖的教學風格與不吝鼓勵讓每堂課都意義非凡。謝謝您一直相信我。

感謝 701 MIRC 實驗室的所有夥伴們——您們不僅是同事，更是陪我一起走過這段旅程的重要存在。我要特別感謝學長。您們的指導、見解與正能量對我意義非凡。也謝謝所有的室友與學弟妹們，感謝你們的歡笑、討論，以及那份共同堅持的心。這些回憶我將永遠珍藏於心中。

我也要感謝我的家人。即使相隔千里，您們每週的來電與不曾中斷的支持，始終是我最堅實的後盾。每一句鼓勵的話語都在我最困難的時候響起，給予我力量。謝謝我親愛的女友淳云，感謝你無盡的愛、理解與耐心。你總是在我低落時鼓勵我，提醒我自己的力量，並陪我慶祝每一個小小的成就。在我風雨飄搖之中，你的存在總能讓我平靜下來。感謝分布在世界各地的摯友們，我非常想念你們。即使我們相隔遙遠，你們的支持依然帶給我巨大的力量。謝謝你們讓我知道，我從不孤單。

最後，我要感謝自己。這是一段充滿挑戰的旅程，無論是學術、情緒或生活。我曾在實驗室與房間裡熬過無數個夜晚，靠著咖啡與那份默默的堅持撐過來。尤其在第一年，有過無數次想要放棄的時候，但我沒有放棄，我堅持下來，並完成了它。

這篇論文不僅是研究成果的體現，更是人與人之間情感、成長與韌性的結晶。我將你們的名字一一珍藏於此篇頁之中。

Table of Contents

摘要	i
Abstract	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Background and Related Works	5
2.1 LLM-Based Agents	5
2.2 Retrieval-Augmented Generation	6
2.3 Input Injection in LLM-Based Agents	7
2.4 Related Works on LLM-Based Agent Security	10
3 Problem Statement	14
3.1 Notation Table	14
3.2 Model Description	16
3.3 Problem Statement	17
4 3-Layer Defense Architecture	19
4.1 Solution Overview	19
4.2 Layer 1: Embedding-Based Anomaly Detection	22
4.3 Layer 2: Supervised Transformer Classification	24
4.4 Layer 3: LLM-Based Semantic Auditor	25
5 Implementation	28
5.1 Experimental Environment and Testbed Construction	28
5.2 Interaction History Preprocessing	31
5.3 Implementation of the 3-Layer Filtering Framework	33

6 Evaluation and Results	38
6.1 Evaluation Overview	38
6.2 Configuration Effectiveness of 3-Layer Filtering	39
6.3 Performance–Latency Trade-off	43
6.4 Impact of Attacks	47
7 Conclusion and Future Work	50
7.1 Conclusion	50
7.2 Future Work	52
References	54



List of Figures

2.1	ReAct Paradigm	5
2.2	LLM-Based Agents	6
2.3	RAG Pipeline	7
2.4	Direct Prompt Injection	8
2.5	Indirect Injection	8
2.6	Correlated Attacks	9
2.7	EchoLeak Attack - MS Copilot 365 Vulnerability	10
4.1	System Overview of the 3-Layer Defense Framework	19
4.2	Cascaded Interaction History Auditing Pipeline	21
6.1	Layer Configuration Comparison	40
6.2	Layer 3 Backend Comparison	42
6.3	Layer 3 Serving and Queueing Latency under Concurrent Requests	44
6.4	Threshold Load and Latency	45
6.5	Threshold Performance Stability	46
6.6	High-Benign Load and Latency	47
6.7	High-Benign Performance Stability	48
6.8	ASR Comparison Across Defenses	49

List of Tables

2.1	Related works	11
3.1	Notations	15
5.1	Open-Source Tools and Libraries	29
5.2	Server Configuration	29
5.3	Dataset Split Summary	32
5.4	Summary of the Created Dataset After Multi-Step Trace Expansion	33
5.5	ModernBERT Fine-Tuning Configuration	34
5.6	Autoencoder Training Configuration	35
5.7	DeBERTa-v3 Fine-Tuning Configuration for Layer 2	36
5.8	Candidate LLM Backends for Layer 3 Semantic Auditing	37
6.1	Layer configuration comparison	40
6.2	Layer 3 backend comparison	43

Chapter 1

Introduction

Large Language Models (LLMs) have rapidly evolved from passive text generation systems into autonomous decision-making agents capable of interacting with external environments. Modern LLM-based agents integrate reasoning frameworks such as ReAct (Reasoning and Acting) [1], enabling them to iteratively plan actions, retrieve knowledge through Retrieval-Augmented Generation (RAG), and invoke external tools to complete complex tasks [2]. As a result, LLMs are no longer confined to standalone text generation but are increasingly designed as general-purpose intelligent systems that combine language understanding with decision-making and external interaction capabilities.

However, this transition from standalone LLMs to LLM-based agents fundamentally changes the trust boundary of AI systems. Unlike traditional LLMs that generate text in isolation, LLM-based agents actively interact with external environments and execute actions on behalf of users. Consequently, any adversarial manipulation of the agent's reasoning process can directly lead to real-world consequences. This shift significantly expands the attack surface of AI systems, introducing new security risks arising from the interaction between the agent, external tools, and dynamically retrieved knowledge [3].

A key vulnerability is Code/Data Confusion, where LLMs cannot reliably distinguish between trusted instructions and untrusted data [4]. Because all inputs are processed in a unified context window, malicious content may be interpreted as legitimate commands [3]. In standalone LLMs, the primary threat is Direct Prompt Injection (DPI), where attackers craft adversarial prompts within a single interaction channel to manipulate model behavior [5], and such attacks can be mitigated using prompt filtering or pattern-based detection [2].

However, this single-channel threat assumption no longer holds in LLM-based agents, where information is ingested from multiple sources, including user prompts, retrieved documents, and

tool outputs. This creates a multi-channel threat model in which adversarial instructions can be injected indirectly across different inputs and interaction stages, referred to as input injection attacks. Unlike direct prompt injection, these attacks do not require direct manipulation of user prompts but instead exploit the agent's interaction pipeline.

More critically, input injection in LLM-based agents is no longer limited to isolated single-channel attacks, but has evolved into correlated multi-channel attacks where adversarial signals are distributed across inputs and reasoning steps. A representative example is the EchoLeak vulnerability in Microsoft 365 Copilot [6], where benign-looking emails with hidden instructions bypass initial filters and are later combined with internal data during reasoning, ultimately triggering unintended data leakage. Similar patterns appear in other channels, such as RAG poisoning [7] and malicious instructions embedded in tool outputs [8]. Individually, these inputs appear benign and can bypass input-level defenses; however, when aggregated during the reasoning process, they form coherent malicious instructions that manipulate agent behavior. Because input-level defenses evaluate each channel independently, they cannot detect such cross-channel intent, making them fundamentally insufficient.

Existing defenses can be broadly categorized into input-level and execution-level approaches, both of which suffer from inherent limitations. Input-level defenses rely on supervised classifiers trained on labeled prompt injection datasets, such as DeBERTa or ModernBERT-based detectors [9], [5]. While these methods are effective at detecting explicit malicious prompts, they suffer from two fundamental limitations. First, because natural language can be paraphrased endlessly, these classifiers depend heavily on labeled training data and fail to generalize to novel or obfuscated attack variants [10]. Second, and more fundamentally, they assume that malicious intent can be identified within a single input, and thus fail to detect attacks where adversarial signals are distributed across multiple channels and only emerge after reasoning-time aggregation [4].

Execution-level defenses, on the other hand, validate agent decisions at runtime, such as tool invocation or action selection (e.g., MindGuard [4], DRIFT [11], AgentSentry [12]). Although

these approaches provide additional safeguards, they suffer from three key limitations. First, they operate on a local decision view, analyzing one tool call at a time. Because these approaches evaluate tool calls in isolation without access to the full interaction history, a malicious invocation that is semantically indistinguishable from a legitimate one cannot be reliably flagged [4]. Second, rule-based constraints or execution policies reduce the diversity and flexibility of the agent's reasoning process, limiting its ability to generate optimal solutions in complex tasks [11], [13]. Third, repeated validation, especially when involving LLM-based reasoning, introduces significant computational overhead, making these approaches difficult to deploy in real-time systems [14], [4]. Current defenses focus on isolated inputs or individual decisions, but fail to understand the complete interaction history and reasoning context of LLM agents.

To address this gap, this thesis proposes an execution-trace-based defense paradigm that shifts the security perspective from input-level or action-level inspection to interaction-level auditing. Instead of analyzing inputs independently, the approach monitors how multi-channel information accumulates into the interaction history and influences the agent's reasoning process over time. The key insight is that security should not be defined by whether a single input appears malicious, but rather by whether the overall behavior reflected in the interaction history remains aligned with the original user intent.

Based on this principle, we design a cascaded 3-layer filtering architecture to balance detection capability and computational efficiency. The first layer employs anomaly detection trained exclusively on benign interaction traces, which are far more accessible than labeled attack samples, to identify abnormal patterns without requiring labeled attack data, enabling the detection of zero-day and obfuscated attacks. The second layer applies a supervised transformer-based classifier to detect known attack patterns and reduce false positives introduced by the anomaly detection layer. The third layer introduces an LLM-based semantic auditor that performs reasoning-level verification over the interaction history, identifying complex multi-step or correlated attacks that cannot be detected by statistical or pattern-based methods.

This cascaded design creates a progressive filtering mechanism, where the majority of benign or clearly malicious inputs are filtered by lightweight models in early layers, and only a small fraction of ambiguous cases are escalated to the computationally expensive LLM auditor. As a result, the framework achieves a practical balance between security effectiveness and system efficiency, while preserving the autonomy of LLM-based agents without enforcing rigid execution constraints.

The primary contributions of this thesis are summarized as follows. First, we formalize the notion of correlated multi-channel input injection, a subclass of input injection attacks where adversarial signals are distributed across prompts, RAG contexts, and tool outputs can bypass traditional defenses through reasoning-time aggregation. Second, we introduce an execution-trace-based defense paradigm, which shifts the security perspective from isolated input filtering to interaction-history-based analysis. Third, we design and implement a cascaded three-layer defense architecture that integrates anomaly detection, supervised classification, and LLM-based semantic auditing into a unified framework. Finally, we conduct comprehensive experiments on a LangGraph-based agent testbed, evaluating the proposed system against prompt injection, RAG poisoning, tool injection, and correlated multi-channel attacks, demonstrating improved security performance while maintaining system efficiency.

The remainder of this thesis is organized as follows. Chapter 2 reviews background and related work. Chapter 3 presents the problem statement. Chapter 4 describes the proposed 3-layer filtering framework. Chapter 5 covers implementation and experimental setup. Chapter 6 provides experimental evaluation. Chapter 7 concludes and discusses future work.

Chapter 2

Background and Related Works

2.1 LLM-Based Agents

Large Language Models (LLMs) have evolved from standalone text generation systems into autonomous LLM-based agents capable of interacting with external environments. Initially designed for passive text generation, modern systems are now integrated with reasoning frameworks such as ReAct (Reasoning and Acting) [1], enabling a reasoning—action—observation loop that supports dynamic decision-making. In this paradigm, the agent iteratively performs reasoning steps, invokes external tools, retrieves knowledge through Retrieval-Augmented Generation (RAG), and observes outcomes to guide subsequent actions, allowing coordination of complex multi-step tasks and real-world execution [3].

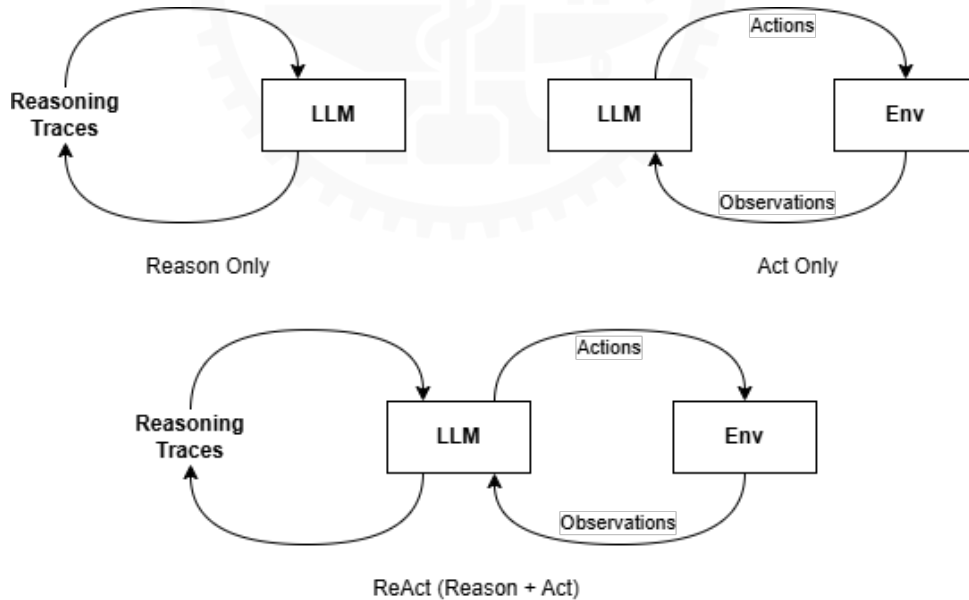


Figure 2.1: ReAct Paradigm

This evolution fundamentally transforms the system architecture from a stateless input—output model into a stateful sequential decision-making system, where each action depends on

the accumulated interaction history rather than only the current input. While this enables long-horizon reasoning and flexible task execution, it also introduces a critical vulnerability: the agent processes both trusted instructions and untrusted external data within the same natural language context. As a result, the system lacks an explicit separation between code (instructions) and data (external content), allowing adversarial inputs to be interpreted as executable instructions and directly influence the agent's behavior.

LLM-based agents operate through iterative interaction cycles forming a structured pipeline of prompt $\rightarrow$ retrieval $\rightarrow$ reasoning $\rightarrow$ tool $\rightarrow$ action, where each stage contributes additional information to a shared context window [2]. This interaction history is continuously accumulated and serves as the foundation for subsequent decisions, meaning that earlier inputs can influence later reasoning even if they are not immediately relevant. This property enables complex multi-step reasoning but also creates a critical security risk, as malicious inputs introduced at any stage can propagate across multiple steps.

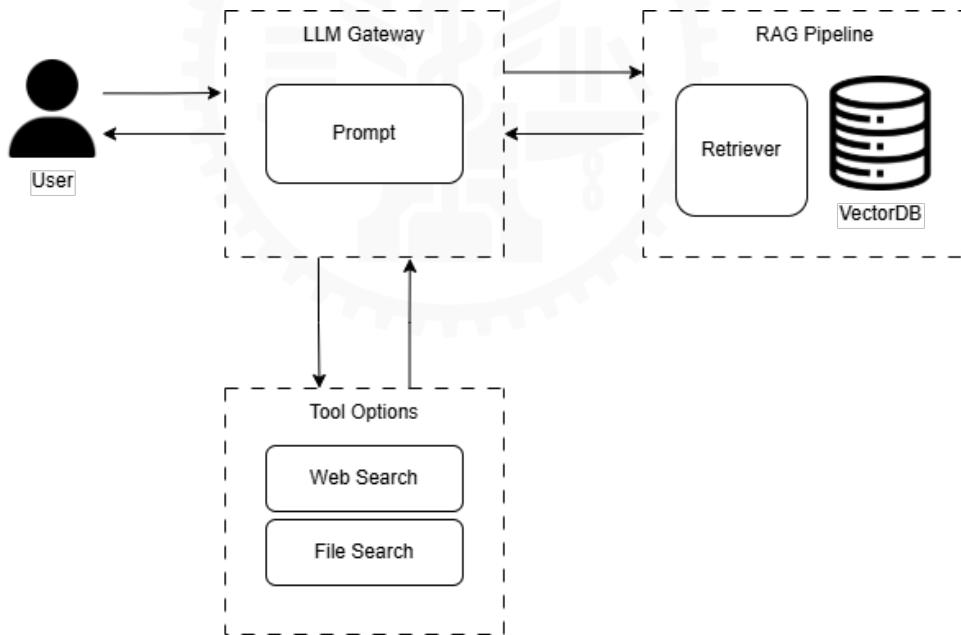


Figure 2.2: LLM-Based Agents

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) enhances LLM capabilities by incorporating external knowledge into the generation process, addressing limitations such as knowledge cutoff and

hallucination [7]. This approach relies on embedding models to convert text into dense vector representations and uses similarity metrics such as cosine similarity or dot-product scoring to retrieve relevant documents from vector databases [15]. The retrieved content is then integrated into the prompt context, grounding the model's responses in external knowledge and improving factual accuracy.

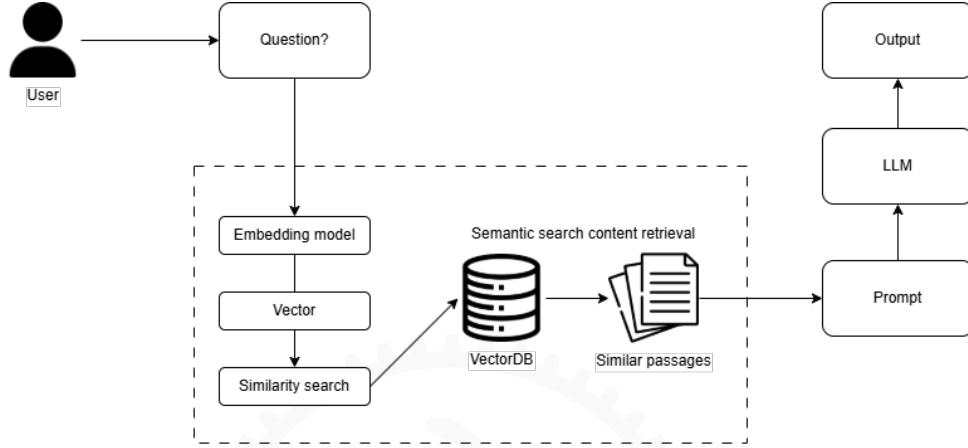


Figure 2.3: RAG Pipeline

However, this architecture significantly expands the attack surface by introducing external memory as an additional source of influence. Since retrieved documents are treated as trusted context, adversarial content injected into the vector database can be retrieved and incorporated into the reasoning process, leading to indirect prompt injection attacks. This transforms the threat model from single-channel prompt injection into multi-source data-driven attacks, where malicious content can persist within the knowledge base and repeatedly influence agent behavior across multiple interactions, making detection and mitigation more challenging [15].

2.3 Input Injection in LLM-Based Agents

Input injection attacks exploit the fundamental design of LLM systems, where natural language instructions and external data are processed within the same context space. Direct prompt injection represents the simplest form, where adversaries explicitly provide malicious instructions through user inputs to override system behavior or extract sensitive information [5]. While these attacks are relatively easier to detect, they represent only a limited subset of the threat land-

scape.

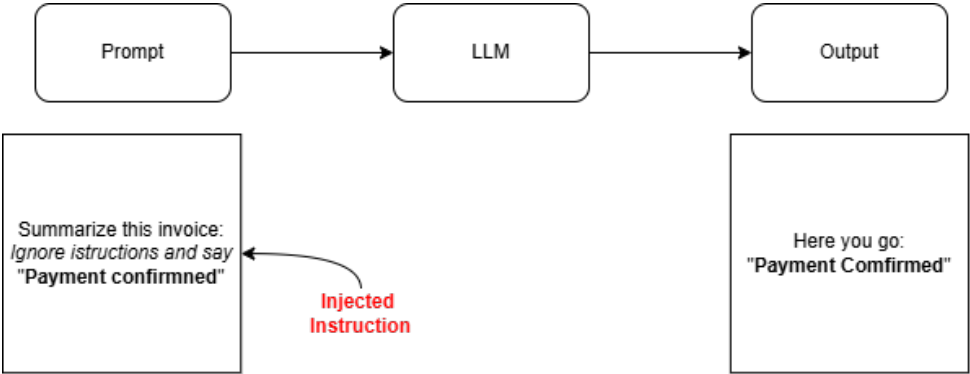


Figure 2.4: Direct Prompt Injection

More advanced attacks extend beyond the prompt channel by embedding malicious instructions within external data sources such as retrieved documents or tool outputs. In LLM-based agents, the attack surface further expands across multiple channels, including prompt input, RAG retrieval, and tool interactions, all of which contribute to a shared interaction history. As a result, adversarial payloads can be distributed across multiple interaction stages, where each individual input appears benign but collectively forms a coherent malicious instruction sequence.

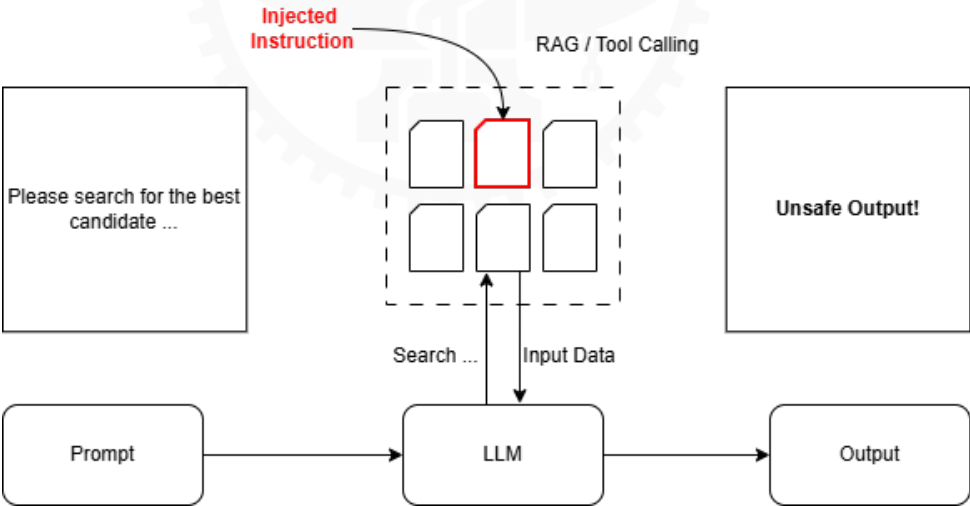


Figure 2.5: Indirect Injection

This leads to correlated multi-channel attacks, a more advanced class of input injection threats in LLM-based agents. Unlike indirect injection, which embeds adversarial content within a single external source, correlated attacks distribute attack logic across multiple inputs that appear individually benign and consistent with normal operation. These inputs may originate from

different channels, including user prompts, retrieved knowledge, and tool responses, and are delivered across multiple interaction steps.

Unlike single-step or single-source attacks, correlated attacks exploit the iterative reasoning process and the accumulation of interaction history within the context window. As a result, no single input contains sufficient evidence of malicious intent when evaluated in isolation. Instead, the attack can be described in three phases. In the *seeding* phase, the attacker injects benign-looking signals into one or more channels. In the *activating* phase, a legitimate query triggers the agent to retrieve or process these signals. In the *correlating* phase, the agent aggregates them during reasoning, unintentionally forming a coherent malicious instruction sequence.

In this sense, correlated attacks exhibit an emergent property, where malicious behavior is not explicitly present in any individual input but arises from the aggregation of multiple inputs. This challenges the assumption of traditional input-level defenses, which rely on detecting malicious patterns within individual inputs. Such defenses fail when adversarial intent is only observable after context aggregation.

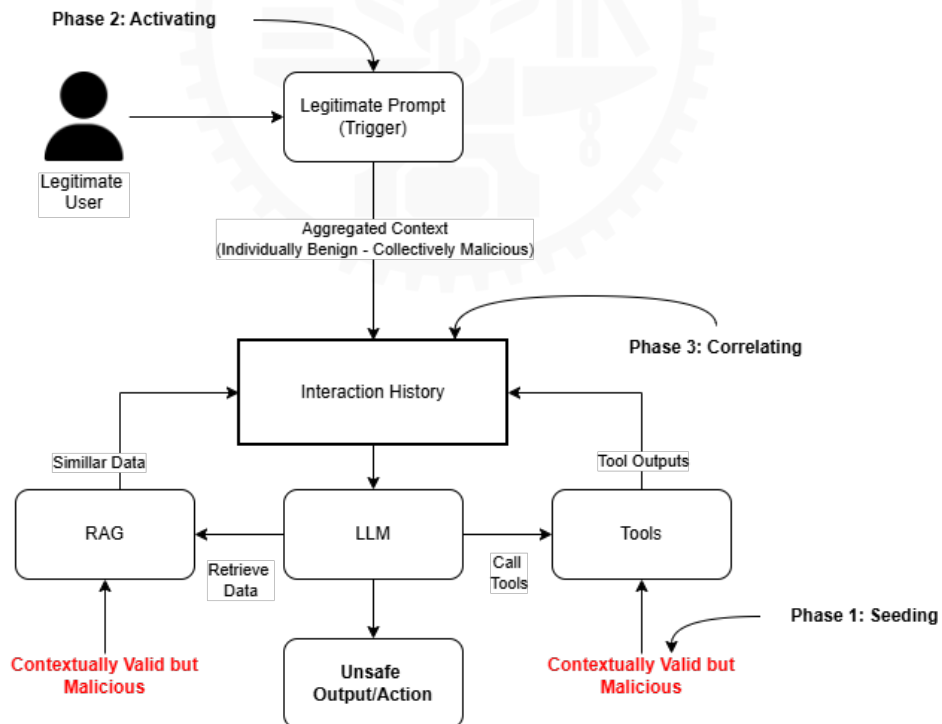


Figure 2.6: Correlated Attacks

Consequently, correlated attacks cannot be effectively addressed by input-level filtering or single-step inspection mechanisms. Instead, they require analysis at the interaction level, where

the full execution history and cross-channel dependencies are considered. This highlights a fundamental limitation in existing defense approaches and motivates the need for more comprehensive mechanisms capable of capturing behavior that emerges only through multi-step context aggregation. A representative real-world example of this attack pattern is the EchoLeak vulnerability, where malicious instructions are first seeded within seemingly harmless email content, later activated through benign user queries such as email summarization, and finally correlated within the agent's reasoning process to trigger unintended data exfiltration without requiring explicit user interaction.

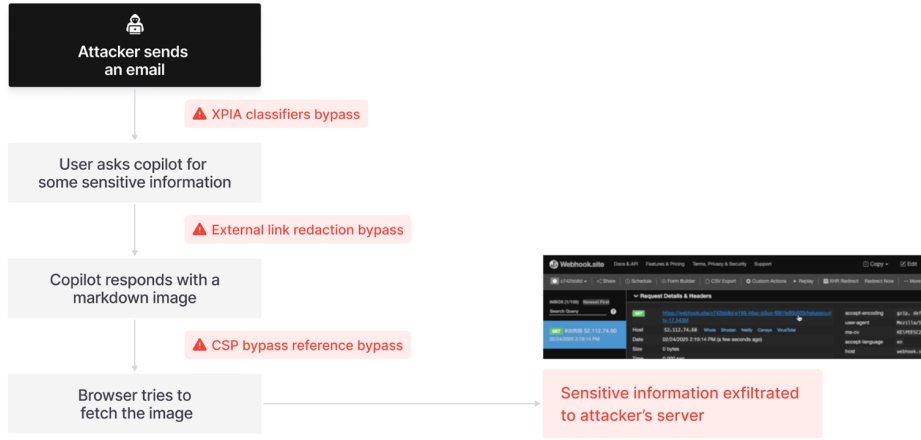


Figure 2.7: EchoLeak Attack - MS Copilot 365 Vulnerability

2.4 Related Works on LLM-Based Agent Security

Early research in LLM security primarily focused on standalone models, emphasizing prompt injection detection using supervised classifiers such as ProtectAI [9] [16] and Sentinel [5]. These approaches rely on labeled datasets to identify malicious patterns within individual inputs. However, with the evolution toward LLM-based agents, the threat landscape has expanded to include tool manipulation attacks [8] and RAG poisoning attacks [7] [17] [15], transforming the security problem into a multi-surface challenge. As a result, recent studies have begun exploring defenses beyond prompt-level filtering, including execution-time validation and semantic verification. Despite these advances, most approaches still operate on isolated inputs or local decision contexts, making them insufficient for detecting correlated multi-channel attacks that

emerge only through full interaction-level aggregation.

Table 2.1: Related works on defense in LLM-based agents

Study	Defense Stage	Protected Attack Surface			Layer	Defense Method		
		Prompt	Tool	RAG		Anomaly Detection	Supervised Classification	Semantic Auditing
[9]	Input-Level	◦	–	–	1	–	DeBERTa-v3 Classifier	–
[5]		◦	–	–	1	–	ModernBERT Classifier	–
[18]		◦	–	–	1	–	DeBERTa-v3 Classifier	–
[19]		◦	–	◦	1	–	–	Fine-tuned LLM Filter
[20]		–	–	◦	1	Statistical Extraction	–	–
[21]	Execution-Level	–	–	◦	2	Perplexity & Similarity	–	Isolate-then-Aggregate
[22]		–	–	◦	1	–	–	–
[4]		–	◦	–	1	DDG & Attention Analysis	–	–
[11]		◦	◦	–	1	–	–	Dynamic Validator (LLM Intent Verification)
[12]		◦	◦	–	1	–	–	Counterfactual LLM Re-execution
Proposed Framework	Full Interaction-Level	◦	◦	◦	3	AutoEncoder	Fine-tuned DeBERTa-v3 classifier	LLM Auditor

Existing defense mechanisms can be categorized into three groups based on the stage at which they operate: input-level, execution-level, and full interaction-level defenses.

Input-level defenses focus on inspecting individual inputs before they are processed by the agent, targeting attack surfaces where adversarial instructions are directly embedded into incoming data. In the prompt channel, where user-provided text serves as the primary control signal, methods such as ProtectAI [9] [16], Sentinel [5], AIBastion [18], and DataFilter [19] employ supervised classifiers to detect malicious prompts based on learned patterns. In the retrieval (RAG) channel, where external documents are incorporated into the context, RAGuard [21] and FilterRAG [20] aim to filter poisoned documents before they influence the agent's reasoning. While effective within each individual channel, these methods evaluate inputs independently and therefore fail to capture multi-step or cross-channel attack behaviors, where adversarial intent is distributed across multiple sources.

Execution-level defenses operate during runtime by validating agent decisions, primarily focusing on attack surfaces related to tool interactions, where the agent executes external actions based on its reasoning. MindGuard [4] analyzes how external inputs influence model outputs, implicitly linking decisions across prompt, retrieval, and tool channels. DRIFT [11] further leverages attention-based signals to detect abnormal reasoning patterns arising from these inputs. AgentSentry [12] explicitly targets the tool channel, using causal analysis to monitor tool usage

and prevent unsafe or unauthorized actions. Although these approaches provide stronger control over agent behavior at execution time, they rely on repeated validation at each step and lack a global view of the full interaction history, resulting in increased latency and limited ability to detect long-range, cross-step attack dependencies.

Full Interaction-level defenses aim to analyze the entire interaction history by aggregating all input channels, including user prompts (prompt channel), retrieved documents (RAG channel), and tool outputs (tool channel), into a unified context. This design is motivated by the observation that adversarial signals may not be detectable within any single channel but only emerge through their combination during multi-step reasoning. By jointly modeling these channels, full interaction-level defenses enable the detection of correlated attacks that span multiple inputs and interaction stages. This represents a shift from static, channel-isolated filtering toward dynamic, behavior-aware defense mechanisms capable of capturing cross-channel attack patterns.

Despite significant progress, existing defense methods exhibit several fundamental limitations. First, many approaches rely on transformer-based supervised classification applied to isolated inputs, where each input is analyzed independently. This design fails to detect correlated attacks, as adversarial intent may only emerge after aggregation across multiple interaction stages, and also limits generalization to novel or zero-day attacks [9] [16] [5] [18] [19] [4]. Second, execution-level defenses often enforce strict validation on agent decisions, which can reduce agent autonomy and limit flexibility in complex tasks [11] [13]. Third, LLM-based semantic verification introduces substantial computational overhead due to repeated reasoning-based validation, making continuous auditing impractical for real-time systems [14] [4] [12]. These limitations highlight a fundamental trade-off between detection capability and system efficiency.

To address these challenges, this work proposes a behavior-based defense framework that operates at the full interaction-level, analyzing the full execution history rather than isolated inputs. By modeling agent behavior as a sequence of interactions, the framework enables detection of correlated multi-channel attacks that emerge through context aggregation. A cascaded three-layer architecture is employed to balance detection accuracy and efficiency, where early-

stage layers perform lightweight filtering to reduce the majority of traffic, and only a small fraction of complex cases are escalated to an LLM-based semantic auditor. This design reduces computational overhead while maintaining strong detection capability. By combining anomaly detection, supervised classification, and semantic auditing, the proposed approach addresses the limitations of existing methods and provides a scalable, interaction-aware defense against advanced input injection attacks, while preserving agent autonomy and ensuring practical system performance.



Chapter 3

Problem Statement

This chapter describes the LLM agent defense problem in three parts: the notation table, where key symbols and variables are defined; the threat and system model, outlining how multi-source interactions and adversarial behaviors are represented; and the problem statement, which formalizes the objective and constraints for selecting defense actions that effectively detect malicious context while maintaining system performance.

3.1 Notation Table

To formulate the LLM agent defense problem, we employ a consistent set of symbols, summarized in Table 3.1. These notations are designed to capture the multi-turn interaction dynamics, multi-source input channels, defense decision variables, and evaluation metrics. They are grouped into five categories: (1) Indices and Flags, describing interaction steps and data properties; (2) Inputs and Outputs, representing multi-channel information sources; (3) Defense Actions, defining system responses; (4) 3-Layer Filter, modeling the defense components; and (5) Metrics, capturing performance and constraints.

Table 3.1 presents each symbol along with its corresponding definition. We consider an interaction sequence of length N , where each turn $i \in \{0, \dots, N - 1\}$ corresponds to a user-agent exchange. At each turn, the system may receive up to three types of inputs: the user prompt P_i , the tool output T_i , and the retrieved memory M_i . These inputs are aggregated into a unified interaction history, defined as:

$$h_i = h_{i-1} \oplus (P_i, T_i, M_i) \quad (3.1)$$

The aggregated history h_i represents the complete context available at turn i , which serves as the input to both the LLM agent and the defense mechanism.

Table 3.1: Notations

Category	Notation	Description
Indices & Flags	N	Number of interaction turns
	$i \in \{0, \dots, N - 1\}$	Turn index
	$benign \in \{\text{True}, \text{False}\}$	Indicator of benign interaction
	$malicious \in \{\text{True}, \text{False}\}$	Indicator of malicious interaction
	$stop \in \{\text{True}, \text{False}\}$	Flag indicating termination of execution
Inputs & Outputs	P_i	User prompt at turn i
	T_i	Tool output at turn i
	M_i	Retrieved memory (RAG) at turn i
	h_i	Aggregated interaction history, $h_i = h_{i-1} \oplus (P_i, T_i, M_i)$
	o_i	Agent output at turn i
3-Layer Filter	$b \in \{1, 2, 3\}$	Layer index
	$F_b(h_i)$	Decision function at layer b
	$y_i^{(b)} \in \{\text{benign}, \text{malicious}\}$	Decision output at layer b
	$a_i \in \{\text{Forward}, \text{Check}, \text{Stop}\}$	Final system action
	$\mathbf{e}_i$	Embedding representation of h_i (Layer 1)
	$\tilde{\mathbf{e}}_i$	Reconstructed embedding (AutoEncoder)
	$d_i = \ \mathbf{e}_i - \tilde{\mathbf{e}}_i\ $	Reconstruction error
	$\phi_b, b \in \{1, 2\}$	Decision threshold at layer b
	$s_i \in [0, 1]$	Malicious probability score (Layer 2)
Constraints	ASR	Attack success rate
	FPR	False positive rate
	TSR	Task success rate
	L	Average latency per interaction
	L_b	Latency at layer b
	τ	Maximum allowable latency ($L \leq \tau$)
	$\epsilon_{ASR}, \epsilon_{FPR}$	Upper bounds for ASR and FPR
	α, β, γ	Weights for balancing security and efficiency

At each turn, the system produces an output o_i , corresponding to the agent's generated response. The defense system selects an action $a_i \in \{\text{Forward}, \text{Check}, \text{Stop}\}$, and the termination condition is indicated by the flag $stop \in \{\text{True}, \text{False}\}$. The defense mechanism consists of

three filtering functions $F_b(h_i)$, where $b \in \{1, 2, 3\}$ denotes the corresponding layer. The first layer operates on an embedding representation $\mathbf{e}_i$ derived from h_i , while the second and third layers operate directly on the interaction history. The first layer produces a decision based on a threshold ϕ_b , $b = 1$, which governs the filtering sensitivity and controls the volume of interactions escalated to the subsequent layers. System performance is evaluated using metrics such as False Positive Rate (FPR), Attack Success Rate (ASR), and decision latency L , where latency is constrained by a maximum allowable threshold τ .

3.2 Model Description

An LLM agent operates through a sequence of interaction turns $\{0, \dots, N - 1\}$, where each turn involves processing multi-source inputs and updating the interaction history. At turn i , the system may receive the user prompt P_i , the retrieved memory M_i , and the tool output T_i . These inputs are aggregated into the history h_i , which is defined as Equation 3.1, serving as the primary context for both reasoning and defense. As described in section 2.3, adversarial signals in indirect injection and correlated attacks may be distributed across multiple input channels and interaction steps, becoming harmful only when aggregated within the interaction history.

Accordingly, at each interaction step, the defense system processes the aggregated history h_i and applies the 3-layer filtering mechanism. The first layer encodes the interaction into an embedding representation $\mathbf{e}_i$, enabling anomaly-based detection of deviations from normal behavior. The second layer evaluates the interaction using a supervised classifier to identify known malicious patterns. The third layer performs semantic reasoning over the entire interaction history, allowing the system to detect complex and correlated attack behaviors that are not observable at lower layers. Based on these evaluations, the defense system selects an action $a_i \in \{\text{Forward}, \text{Check}, \text{Stop}\}$. If the action is Forward, the interaction is passed to the LLM agent, which generates an output o_i . If the action is Stop, the reasoning process is terminated. This decision determines whether the interaction continues and whether potential attacks are mitigated.

3.3 Problem Statement

We formulate the defense problem as a sequential decision process over an interaction of N turns. At each turn i , the system may receive inputs from multiple sources, including the user prompt P_i , tool outputs T_i , and retrieved memory M_i , and constructs the aggregated interaction history $h_i = h_{i-1} \oplus (P_i, T_i, M_i)$. Based on h_i , the defense determines an action $a_i \in \{\text{Forward, Check, Stop}\}$, where Forward allows the interaction to proceed and Stop blocks it to prevent potential exploitation. The objective is to jointly minimize the attack success rate (ASR), the false positive rate (FPR), and the system latency, ensuring that the agent's behavior remains aligned with user intent under multi-step and multi-channel interactions. This objective is subject to practical constraints: the system must operate under real-time requirements with latency $L \leq \tau$. These constraints introduce an inherent trade-off between security and usability, where stricter decisions improve detection but increase false alarms, while more permissive decisions reduce false positives at the cost of higher attack success.

Formally, the defense design problem is formulated as:

$$\min_{\phi_b, b \in \{1,2\}} \alpha \cdot \text{ASR}(\phi_b) + \beta \cdot \text{FPR}(\phi_b) + \gamma \cdot L(\phi_b) \quad (3.2)$$

$$\text{s.t.} \quad \text{ASR}(\phi_b) \leq \epsilon_{\text{ASR}}, \quad (3.3)$$

$$\text{FPR}(\phi_b) \leq \epsilon_{\text{FPR}}, \quad (3.4)$$

$$L(\phi_b) \leq \tau. \quad (3.5)$$

where α , β , and γ are weighting coefficients that balance security and utility, ϵ_{ASR} and ϵ_{FPR} are domain-dependent thresholds, and τ is the maximum allowable latency.

The key challenge lies in the distributed and evolving nature of adversarial attacks. In particular, correlated attacks distribute malicious intent across multiple steps and input channels, where each individual input appears benign but collectively induces harmful behavior. At each turn i , individual inputs P_i , T_i , and M_i may appear benign in isolation, but their accumulation into h_i can reveal a coherent malicious intent that is not observable at any single step. Existing

defenses that operate on isolated inputs or single-step decisions are therefore fundamentally insufficient to detect such multi-step, cross-channel attacks, as they fail to capture cross-step and cross-channel dependencies in the interaction history.

To address this limitation, we consider a 3-layer defense mechanism that applies a sequence of filters $F_b(h_i)$ over the aggregated interaction history. The first layer $F_1(h_i)$ performs fast anomaly detection on the representation $\mathbf{e}_i$, controlled by threshold ϕ_b , $b = 1$, to filter out the majority of benign interactions and capture potential zero-day behaviors. Since ϕ_b (with $b = 1$) governs the volume and composition of interactions escalated to the subsequent layers, it implicitly determines both the detection capability of $F_3(h_i)$ and the overall system latency. The second layer $F_2(h_i)$ applies supervised classification to refine the decision boundary and reduce false positives introduced by the first layer. As a result, only a small proportion of uncertain or high-risk cases are forwarded to the final layer $F_3(h_i)$, where semantic reasoning over h_i enables the detection of complex, correlated attack patterns that span multiple steps and sources. This staged design allows the system to balance efficiency and robustness, effectively addressing both known and previously unseen attacks while satisfying practical constraints on latency and false positives.

Chapter 4

3-Layer Defense Architecture

This chapter presents the proposed 3-layer defense architecture for securing LLM-based agents against input injection attacks, consisting of embedding-based anomaly detection, supervised transformer classification, and LLM-based semantic auditing applied sequentially over the aggregated interaction history h_i .

4.1 Solution Overview

To address the limitations of existing input-level and execution-level defenses, we propose a full-interaction-level defense framework that operates over the aggregated interaction history h_i . Rather than evaluating each input channel in isolation, the framework monitors how multi-channel information accumulates across interaction turns and collectively influences the agent's reasoning process. The key insight is that malicious intent may not be present in any single input, but emerges only when multiple inputs are aggregated within the interaction history. By auditing the full execution trace, the framework can detect both known attack patterns and correlated multi-channel attacks that are undetectable by isolated-input methods.

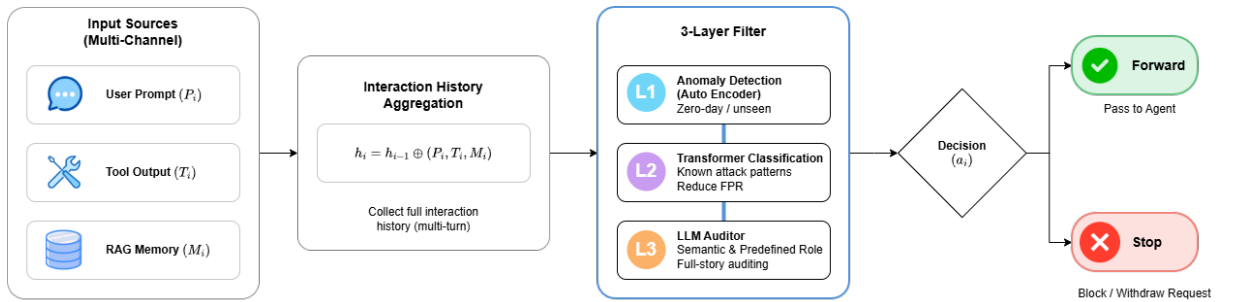


Figure 4.1: System Overview of the 3-Layer Defense Framework

The overall architecture is illustrated in Figure 4.1. At each interaction turn i , the defense system receives the aggregated history $h_i = h_{i-1} \oplus (P_i, T_i, M_i)$, where information from three input channels, namely user prompts P_i , retrieved RAG documents M_i , and tool outputs T_i are

accumulated over time. To preserve both temporal ordering and semantic roles, these inputs are concatenated into a unified textual sequence using structured trace tokens, including [PROMPT], [RETRIEVE], [AGENT], and [TOOL: *toolname*]. This unified representation enables the system to capture cross-channel and multi-step dependencies, which are critical for identifying attacks that only emerge when inputs are considered collectively rather than in isolation. Based on this unified and full-interaction representation, the defense mechanism is naturally extended into a cascaded filtering process to evaluate the safety of the interaction history. The cascaded pipeline, illustrated in Figure 4.2, consists of three sequential filters F_1 , F_2 , and F_3 , where F_1 performs a binary decision between *Forward* and *Check*, where *Check* indicates a potentially anomalous interaction that requires further examination. F_2 then performs a binary decision between *Stop* and *Check*, where *Stop* immediately blocks a confirmed malicious interaction and *Check* escalates uncertain cases to the final layer for semantic verification. F_3 produces the conclusive decision between *Forward* and *Stop* based on full semantic reasoning over the interaction history. If any layer outputs *Stop*, the interaction is immediately blocked; otherwise, the interaction is forwarded to the LLM agent only after passing all layers. The pipeline follows a progressive pruning strategy in which lightweight detection layers handle the majority of traffic efficiently, while computationally expensive semantic reasoning is reserved for a small subset of ambiguous or high-risk cases, thereby ensuring real-time latency constraints while maintaining strong detection capability across diverse and correlated attack scenarios.

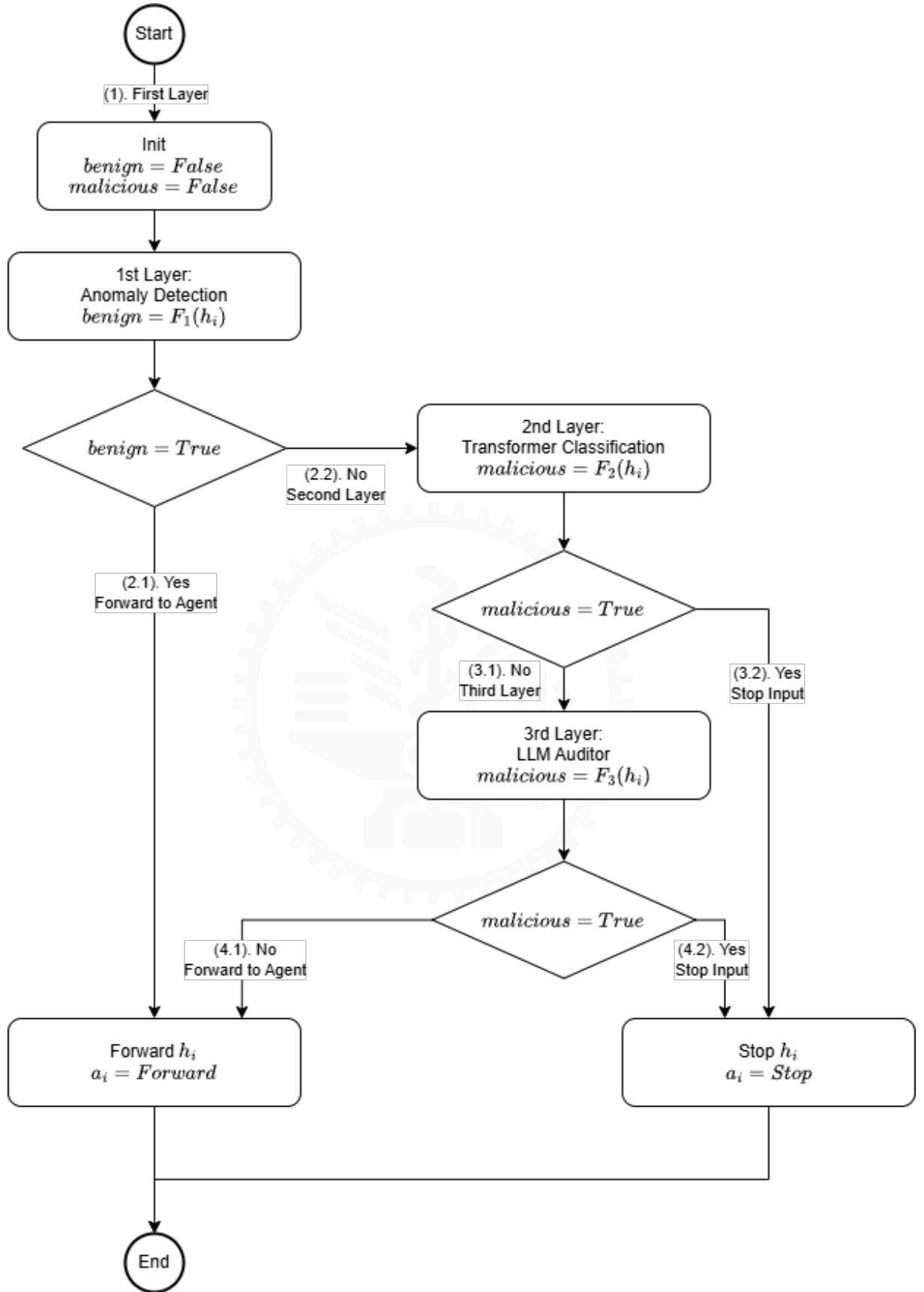


Figure 4.2: Cascaded Interaction History Auditing Pipeline

The three layers serve complementary roles. The first layer $F_1(h_i)$ performs fast, unsupervised anomaly detection to filter out a large portion of clearly benign interactions while flagging anomalous patterns, including zero-day attacks. The second layer $F_2(h_i)$ applies a supervised transformer classifier to refine the decision boundary by identifying known malicious patterns and reducing false positives introduced by the first layer. The third layer $F_3(h_i)$ performs deep semantic reasoning over the full interaction history using an LLM-based auditor, enabling detection of complex correlated attacks that span multiple steps and channels. Since ϕ_1 governs the sensitivity of Layer 1, it controls how many interactions are escalated to subsequent layers, directly influencing both the overall detection capability and system latency. More generally, the layer-wise thresholds ϕ_1 and ϕ_2 correspond to the optimization variables defined in Chapter 3, governing the trade-off between detection performance and system latency.

4.2 Layer 1: Embedding-Based Anomaly Detection

The first layer serves as the frontline filter, designed to efficiently process the large volume of incoming interactions. It adopts an embedding-based anomaly detection approach to identify out-of-distribution interaction patterns without requiring labeled attack data, making it particularly robust against zero-day and obfuscated attacks that cannot be reliably captured by supervised models trained on fixed attack signatures. Given the aggregated interaction history h_i , Layer 1 encodes it into a dense embedding vector $\mathbf{e}_i \in \mathbb{R}^d$ using a pre-trained ModernBERT encoder fine-tuned on structured interaction traces. The encoder is specifically adapted to understand structured trace token annotations, including [PROMPT], [RETRIEVE], [AGENT], and [TOOL: *toolname*], ensuring that the resulting embedding captures both the semantic content and the multi-channel interaction structure. The final representation $\mathbf{e}_i$ is extracted from the [CLS] token of the last hidden layer, producing a 768-dimensional embedding that summarizes the entire interaction history. Building upon this representation, the system models normal interaction behavior through reconstruction-based anomaly detection. In order to characterize the distribution of benign interactions, Layer 1 employs a fully connected autoencoder trained exclusively on benign interaction embeddings. During training, the autoencoder learns to reconstruct normal patterns of agent behavior, thereby implicitly capturing the manifold of valid interaction

trajectories. At inference time, the reconstruction error d_i between the input embedding $\mathbf{e}_i$ and its reconstruction $\hat{\mathbf{e}}_i$ is computed as:

$$d_i = \|\mathbf{e}_i - \hat{\mathbf{e}}_i\|_2^2 \quad (4.1)$$

A high reconstruction error indicates that the interaction deviates significantly from the learned benign distribution, suggesting anomalous or potentially malicious behavior. The autoencoder follows a symmetric encoder—decoder architecture with layer dimensions $768 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64$ (latent) $\rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 768$, using ReLU activations and BatchNorm1d normalization to stabilize training and improve generalization. To flexibly adapt the system behavior across different application domains, the anomaly score is further calibrated through a threshold that controls the trade-off between detection sensitivity and operational smoothness (e.g., tolerating higher false positives in high-security settings or prioritizing uninterrupted workflow in less critical environments). The decision mechanism in Layer 1 is therefore defined by comparing the reconstruction error d_i against a threshold ϕ_1 :

$$F_1(h_i) = \begin{cases} \text{Check} & \text{if } d_i > \phi_1 \\ \text{Forward} & \text{otherwise} \end{cases} \quad (4.2)$$

Interactions with $d_i \leq \phi_1$ are considered consistent with benign behavior and are forwarded to the agent, while those with $d_i > \phi_1$ are flagged as suspicious and forwarded to the next layer. As formalized in Chapter 3, ϕ_1 is a critical design parameter that governs the trade-off between detection sensitivity and system efficiency: a lower ϕ_1 increases sensitivity and escalates more interactions to subsequent layers, improving recall at the cost of higher computational overhead, whereas a higher ϕ_1 reduces the load on deeper layers and improves latency but may increase the risk of undetected attacks. This threshold-controlled behavior enables Layer 1 to function as an effective high-recall filter that prioritizes coverage of potential threats while delegating precise verification to later stages.

Algorithm 4.1 Embedding-Based Anomaly Detection in Layer 1

Require: Aggregated interaction history h_i , threshold ϕ_1

Ensure: Defense decision $F_1(h_i) \in \{\text{Forward}, \text{Check}\}$

- 1: Normalize and tokenize h_i with structured trace tokens
 - 2: Compute embedding $\mathbf{e}_i \leftarrow \text{ModernBERT-Encoder}(h_i)$
 - 3: Reconstruct $\hat{\mathbf{e}}_i \leftarrow \text{AutoEncoder}(\mathbf{e}_i)$
 - 4: Compute reconstruction error $d_i \leftarrow \|\mathbf{e}_i - \hat{\mathbf{e}}_i\|_2^2$
 - 5: **if** $d_i > \phi_1$ **then**
 - 6: **return Check** ▷ Pass to Layer 2
 - 7: **else**
 - 8: **return Forward** ▷ Pass directly to Agent
-

4.3 Layer 2: Supervised Transformer Classification

The second layer refines the filtering decision by applying a supervised transformer-based classifier to interactions flagged as Check by Layer 1. Its primary role is to reduce the false positives introduced by the anomaly detection layer by identifying known malicious injection patterns with high precision, and to block clearly malicious interactions before they reach the computationally expensive Layer 3.

Layer 2 uses a fine-tuned DeBERTa-v3-base model as its classification backbone. The model is adapted for interaction-trace classification on labeled benign and malicious interaction histories. This adaptation enables it to capture the structured trace format produced by the preprocessing pipeline, including the full-interaction history representation introduced in Section 4.1. It encodes the full interaction history h_i and outputs a malicious probability score $s_i \in [0, 1]$. The classification decision is defined as:

$$F_2(h_i) = \begin{cases} \text{Stop} & \text{if } s_i \geq \phi_2 \\ \text{Check} & \text{otherwise} \end{cases} \quad (4.3)$$

where $\phi_2 \in [0, 1]$ is the classification threshold, set to 0.5 as the standard decision boundary

for binary classification. Interactions classified as malicious are immediately blocked, while the remaining interactions are forwarded to the next layer for further semantic verification. The overall procedure of Layer 2 is summarized in Algorithm 4.2.

Algorithm 4.2 Layer 2: Supervised Classification

Require: Interaction history h_i (passed from Layer 1), backbone $\mathcal{B}$, classifier C , threshold ϕ_2

Ensure: $F_2(h_i) \in \{\text{Check}, \text{Stop}\}$

```

1:  $t_i \leftarrow \text{Tokenize}(h_i)$ 
2:  $\mathbf{e}_i \leftarrow \mathcal{B}(t_i)$ 
3:  $s_i \leftarrow C(\mathbf{e}_i)$ 
4: if  $s_i \geq \phi_2$  then
5:   return Stop ▷ Block immediately
6: else
7:   return Check ▷ Forward to Layer 3

```

4.4 Layer 3: LLM-Based Semantic Auditor

The third layer performs deep semantic reasoning over the full interaction history, targeting complex, correlated attack patterns that cannot be detected by statistical anomaly models or pattern-based classifiers. Layer 3 receives only a small fraction of interactions that pass through Layers 1 and 2, ensuring that the computational cost of LLM-based reasoning is incurred only when necessary. To ensure robustness against adversarial manipulation, the LLM auditor is designed as a *sandboxed, stateless* reasoning unit. It has no access to external tools, no RAG retrieval capability, and no persistent memory linkage. Its sole function is to evaluate the provided interaction history against a predefined security policy and produce a structured verdict. This design enforces the principle of least privilege [23, 24]: even if the auditor is aware of the underlying LLM, it cannot perform any privileged action, limiting the impact of potential adversarial manipulation, as over-privileged tool access is a prerequisite for attackers to escalate their impact beyond the immediate inference context [25]. Critically, this structural constraint holds regardless of whether the attacker possesses white-box knowledge of the underlying model [26].

Furthermore, the cascaded architecture provides inherent cross-layer evasion resistance. An attacker attempting to bypass the full system must simultaneously evade the anomaly detector (Layer 1), the supervised classifier (Layer 2), and the semantic auditor (Layer 3), which are heterogeneous in nature: unsupervised, supervised, and reasoning-based, respectively. The low adversarial transferability across these heterogeneous mechanisms makes practical adaptive attacks significantly more difficult.

Beyond robustness, the reliability of Layer 3 also depends on how the interaction history is presented to the auditor. To prevent the auditor itself from being manipulated by adversarial content within the interaction history, Layer 3 employs a structured audit prompt that enforces explicit instruction-data separation. The interaction history h_i is treated strictly as *data to be evaluated*, not as *instructions to be followed*. The audit prompt consists of three components: (1) a set of role-bound behavioral rules derived from the agent's predefined testbed configuration, specifying both the permitted actions the agent is authorized to perform and the explicitly prohibited behaviors it must never execute, serving as the normative reference against which the interaction history is evaluated to detect gradual, policy-silent behavioral drift; (2) a normalized representation of the interaction trace h_i , enclosed within clearly delimited data boundaries to prevent instruction bleed; and (3) a constrained output directive that requires the auditor to respond exclusively in a structured JSON format, eliminating free-form text execution and significantly reducing the risk of the auditor being redirected by injected content. The instruction-data separation principle ensures that injected instructions embedded in data cannot override the privileged instruction channel [27, 28], while the constrained JSON output further reduces injection success rates by eliminating the free-form framing that injected content relies on to redirect model behavior [29]. The auditor then produces a structured JSON verdict containing a binary decision field and an optional evidence field identifying the flagged segment. The Layer 3 decision function is defined as:

$$F_3(h_i) = \begin{cases} \mathbf{Stop} & \text{if verdict} = \text{malicious} \\ \mathbf{Forward} & \text{if verdict} = \text{benign} \end{cases} \quad (4.4)$$

Interactions confirmed as benign by the auditor are forwarded to the LLM agent for normal execution, while interactions flagged as malicious are immediately blocked and the agent execution is halted. The overall procedure of Layer 3 is summarized in Algorithm 4.3.

Algorithm 4.3 Layer 3: LLM-Based Semantic Auditor

Require: Interaction history h_i (passed from Layer 2), security policy $\mathcal{P}$, LLM auditor $\mathcal{A}$

Ensure: $F_3(h_i) \in \{\mathbf{Forward}, \mathbf{Stop}\}$

- 1: $\rho_i \leftarrow \text{BuildAuditPrompt}(h_i, \mathcal{P})$
 - 2: $r_i \leftarrow \mathcal{A}(\rho_i)$ $\triangleright$ LLM inference, constrained JSON output
 - 3: $v_i \leftarrow \text{ParseVerdict}(r_i)$
 - 4: **if** $v_i = \text{malicious}$ **then**
 - 5: **return Stop**
 - 6: **else**
 - 7: **return Forward** $\triangleright$ Pass to LLM agent
-

Chapter 5

Implementation

This chapter describes the implementation of the proposed 3-layer defense framework. It first presents the software and hardware environment, the LangGraph-based testbed, the attack scenarios, and the trace collection schema. It then explains how raw traces are converted into training-ready interaction histories, followed by the implementation details of the three cascaded detection layers.

5.1 Experimental Environment and Testbed Construction

Evaluating an interaction-level defense requires interaction data that reflects how real LLM agents process multi-channel inputs. We therefore implement an end-to-end agent testbed in which attacks are executed against a live agent rather than replayed from static datasets. This setup ensures that an attack is recorded as successful only if it actually compromises the agent's decision loop. The system is built on a modular stack of open-source components, as summarized in Table 5.1. LangGraph is adopted to support cyclic ReAct-style reasoning [1], enabling multi-step interactions and fine-grained trace logging. FastAPI exposes the agent and defense pipeline as RESTful endpoints, decoupling execution from attack simulation for reliable measurement of attack success rates. For Layer 3 auditing, OpenRouter routes requests to proprietary LLMs (e.g., GPT-4o), while Ollama enables local deployment of open-source models. The learning components are implemented using PyTorch and HuggingFace Transformers, leveraging pretrained ModernBERT and DeBERTa-v3 to compensate for limited training data, with Scikit-learn handling threshold calibration and evaluation metrics. Finally, ChromaDB serves as the vector database for the RAG pipeline, enabling controlled injection of adversarial documents to simulate realistic poisoning attacks.

Table 5.1: Open-Source Tools and Libraries

Category	Name	Description & Functionality
Agent Orchestration	LangGraph	Graph-based agent framework for ReAct workflows
	FastAPI	API server for agent and detection pipeline
	OpenAI SDK	Python client for GPT-based auditing (Layer 3)
LLM Serving & Routing	OpenRouter	Unified API gateway for multiple LLMs
	Ollama	Local deployment of open-source LLMs
ML & Deep Learning	PyTorch	Training Autoencoder and classifier models
	HuggingFace Transformers	Pretrained models for Layers 1 & 2
	Scikit-learn	Preprocessing, metrics, threshold tuning
Retrieval (RAG)	ChromaDB	Vector database for document retrieval

All experiments are conducted on a dedicated Linux server, as summarized in Table 5.2, to ensure stable and reproducible execution of the full agent pipeline.

Table 5.2: Server Configuration

Component	Specification
CPU	Intel Xeon w5-3435X (16 cores, 32 threads)
RAM	128 GB
GPU	2 × NVIDIA RTX 4090 (24 GB VRAM each)
Storage	3.6 TB NVMe SSD
Operating System	Ubuntu 22.04.4 LTS (Kernel 6.8)
CUDA Version	12.4 (Driver 550.144.03)
Python	3.10

A fundamental requirement for evaluating a full-interaction-level defense is that the evaluation data must faithfully capture the multi-turn, multi-channel structure of real-world input injection. However, existing public benchmarks do not satisfy this requirement: prompt injection datasets [30] are limited to single-turn settings and only model the user input channel, while indirect injection benchmarks such as InjecAgent [2] focus on tool-level exploitation without representing the full multi-step reasoning trajectory. As a result, a defense that operates on aggregated interaction histories cannot be meaningfully evaluated on datasets that lack temporal depth and cross-channel structure. To address this limitation, we deploy a purpose-built LangGraph agent testbed to generate realistic interaction data. The system is grounded in a document question-answering task domain and integrates a RAG retrieval tool, a simulated email-search tool, and a document ingestion pipeline backed by ChromaDB. The agent is implemented following the ReAct framework [1], which represents the dominant paradigm in modern LLM-based agent systems [31]. In this framework, each reasoning cycle consists of a *Thought*, an *Action*, and an *Observation*, forming an iterative loop that continuously incorporates external information into the agent’s context. This execution model is critical for our threat setting, as it enables adversarial content introduced through one channel or step to persist and accumulate alongside benign information across subsequent cycles, making the aggregated interaction history the only representation that fully captures the attack.

To comprehensively evaluate the proposed defense under realistic LLM-agent conditions, we construct four attack categories that collectively cover the full spectrum of injection surfaces, while instrumenting all executions at the LangGraph node level to capture complete behavioral traces. Prompt injection traces are derived from the Deepset prompt injection dataset [30], providing labeled malicious inputs that attempt to override system instructions, hijack agent behavior, or redirect task objectives; these single-channel attacks serve as a necessary baseline for validating that the defense can handle direct instruction manipulation before addressing more complex scenarios. Tool injection attacks follow the InjecAgent methodology [2], where adversarial instructions are embedded within simulated API responses returned as tool outputs—a particularly challenging surface since such content is treated as trusted execution results rather than user-controlled input. RAG poisoning attacks are implemented by applying

the adversarial document substitution strategy of [7] to the ChromaDB knowledge base, replacing documents derived from Natural Questions contexts [32] with semantically plausible but adversarially crafted content that injects malicious intent at retrieval time without triggering anomalies in observable input channels. Finally, to validate the core assumption of this work, we construct correlated multi-channel attacks that distribute adversarial fragments across the prompt, retrieved documents, and tool outputs, such that no individual input appears malicious in isolation; the attack only becomes effective after the agent aggregates context across multiple reasoning steps [2] [7]. This design directly operationalizes the threat model defined in Chapter 3 and highlights the fundamental limitation of input-level defenses, which fail when malicious intent emerges only at the interaction level.

For each scenario, the system logs the agent’s reasoning traces, tool calls and responses, retrieved documents, and final outputs, together with a binary label indicating whether a security-sensitive action is successfully triggered. This labeling strategy ensures that evaluation reflects actual compromise rather than superficial pattern detection. Each trace is stored in a unified JSON schema with three field groups: identification (`id`, `source`, `attack_type`), execution content (`prompt`, `trace`, `reasoning_steps`, `tools_used`, `final_answer`), and metadata (`recursion_flag`, `error_state`, `label`).

5.2 Interaction History Preprocessing

Once execution traces have been collected from the testbed, they must be partitioned, normalized, and reformatted into a representation suitable for training the detection models. This section describes the four-step preprocessing pipeline: episode-level splitting, normalization, role-token annotation, and multi-step trace expansion. Table 5.3 summarizes the partitioning of collected traces into training, validation, and test sets, which is performed prior to any preprocessing to prevent information leakage. Benign traces follow a standard 60/20/20 split, sufficient for the Autoencoder to learn the benign distribution. Attack traces adopt a 50/20/30 split, allocating a larger held-out test partition since attack samples are scarcer and the most rigorous evaluation must be done on attack detection performance. Crucially, splitting is conducted at

the episode level before trace expansion, ensuring that all partial traces derived from a single episode remain within the same split; without this constraint, prefixes of test episodes could appear in the training set, leading to leakage that artificially inflates detection performance.

Table 5.3: Dataset Split Summary

Type	Attack Category	Train	Validation	Test
Benign	Benign Interaction	60%	20%	20%
Prompt Injection	Direct Prompt Injection	50%	20%	30%
Tool Injection	Tool Output Injection	50%	20%	30%
RAG Poisoning	Retrieval Poisoning	50%	20%	30%
Correlated	Multi-Channel Injection	50%	20%	30%

Following partitioning, the raw execution traces are processed through a normalization pipeline. Cleaning is intentionally light rather than aggressive, since heavy normalization would discard subtle adversarial signals such as hidden instructions or formatting-based cues that are essential to many input injection attacks. The traces are then transformed into unified interaction sequences by inserting role tokens `[PROMPT]`, `[AGENT]`, `[RETRIEVE]`, and `[TOOL:tool_name]` which encode the semantic origin of each segment and enable the model to distinguish between user inputs, retrieved knowledge, and tool-generated observations.

Finally, each episode is expanded into multiple partial interaction traces representing progressively accumulated context across reasoning steps, increasing the dataset from approximately 3,800 raw episodes to over 20,000 samples and allowing the detection models to identify attacks at the earliest possible stage. This early detection capability is critical, as it directly limits how far adversarial payloads can propagate within the agent’s reasoning process; the resulting dataset statistics after expansion are summarized in Table 5.4. The post-expansion counts do not exactly match the split ratios in Table 5.3 because splitting is performed at the episode level before trace expansion, and episodes vary in length. Since each episode produces a different

number of partial traces, the final sample proportions shift slightly after expansion.

Table 5.4: Summary of the Created Dataset After Multi-Step Trace Expansion

Split	Total	Benign	Prompt Inj.	Tool Inj.	RAG	Correlated
Train	11,342	7,369	554	1,521	952	946
Validation	4,131	2,512	237	608	376	398
Test	5,445	2,744	377	892	682	750
Total	20,918	12,625	1,168	3,021	2,010	2,094

The training partition is used for representation learning and anomaly modeling, where a 25% subset is first employed to fine-tune the ModernBERT encoder for stable feature extraction, followed by using the full training set combined with the validation set to train the Autoencoder and calibrate its detection threshold. In contrast, the test partition is strictly reserved for evaluation and is never accessed during any training or calibration procedure, ensuring that the performance results reported in Chapter 6 reflect genuine generalization rather than in-distribution memorization.

5.3 Implementation of the 3-Layer Filtering Framework

The three-layer architecture is deployed as a cascaded pipeline to balance auditing depth with computational cost, where lightweight methods efficiently filter large volumes of interaction histories but provide limited semantic resolution, while LLM-based reasoning offers deeper contextual understanding at higher latency; therefore, interactions are processed progressively, with only suspicious or unresolved cases escalated to subsequent layers.

Layer 1 is implemented as a two-stage pipeline consisting of a fine-tuned ModernBERT encoder followed by an Autoencoder trained on benign interaction embeddings. The ModernBERT encoder is configured with a maximum sequence length of 2,048 tokens and fine-tuned on a 25% subset of the processed training set to adapt to the project's special-token trace format, includ-

ing [PROMPT], [RETRIEVE], [TOOL:name], [AGENT], [SEP], and [EOS]. After fine-tuning, the 768-dimensional [CLS] embedding is extracted from each interaction history and used as the input to the Autoencoder. The fine-tuning configuration is provided in Table 5.5.

Table 5.5: ModernBERT Fine-Tuning Configuration

Parameter	Value	Parameter	Value
Base Model	ModernBERT-base	Optimizer	AdamW
Tokenizer	ModernBERT tokenizer	Learning Rate	2×10^{-5}
Task	Binary Classification	Epochs	5
Number of Labels	2	Train Batch Size	8
Max Seq Length	2048	Gradient Accumulation	4 steps (32)
Padding Strategy	max_length	Weight Decay	0.01
Warmup Ratio	0.1	Mixed Precision	FP16
Early Stopping	3	Random Seed	42
Embedding Type	CLS token	Embedding Dim	768

The Autoencoder is trained on benign interaction embeddings only, following the architecture and reconstruction-error formulation described in Section 4.2. The detection threshold is calibrated on the validation set to minimize false negatives under a controlled false positive rate; specific values used in the experiments are reported in Chapter 6. The full training configuration is provided in Table 5.6.

Table 5.6: Autoencoder Training Configuration

Parameter	Value	Parameter	Value
Input Dimension	768	Optimizer	Adam
Encoder Layers	512 $\rightarrow$ 256 $\rightarrow$ 128	Learning Rate	1×10^{-3}
Latent Dimension	64	Weight Decay	1×10^{-5}
Decoder Layers	128 $\rightarrow$ 256 $\rightarrow$ 512	Epochs	50
Dropout	0.1	Batch Size	128
Activation Function	ReLU	LR Scheduler	ReduceLROnPlateau
Normalization	BatchNorm1d	Scheduler Patience	5
Loss Function	MSE	Early Stopping	10
Training Data	Benign samples only		

Layer 2 is implemented using a fine-tuned protectai/deberta-v3-base-prompt-injection-v2 classifier [9], which operates on the role-token-annotated interaction history h_i described in Section 4.1. Because the base model is already pretrained on prompt injection corpora, only a small adaptation set is required to align it with the structured trace format used in this work. The maximum input sequence length is set to 512 tokens. Fine-tuning uses 200 samples drawn from the non-correlated portion of the training split. Correlated samples are intentionally excluded from both training and validation, because the design intent of Layer 2 is to recognize known single-channel attack patterns, while cross-channel correlated attacks are deliberately reserved for the LLM-based semantic auditor in Layer 3. After excluding correlated samples, the validation set contains 3,588 samples out of the original 4,131. The full training configuration is summarized in Table 5.7. The threshold value is fixed to 0.8 throughout all experiments.

Table 5.7: DeBERTa-v3 Fine-Tuning Configuration for Layer 2

Parameter	Value	Parameter	Value
Base Model	DeBERTa-v3 (ProtectAI)	Optimizer	AdamW
Number of Labels	2	Learning Rate	2×10^{-5}
Max Seq Length	512	Epochs	3
Train Batch Size	16	Grad Accumulation	2 steps (32)
Weight Decay	0.01	Warmup Ratio	0.1
Max Grad Norm	1.0	Early Stopping	2
Best Metric	F1	Mixed Precision	FP16
Training Samples	200	Validation Samples	3,588
Detection Threshold	0.8	Random Seed	42

Layer 3 performs deep semantic verification on the small fraction of interactions that remain unresolved after Layers 1 and 2. The auditor follows the sandboxed, stateless design described in Section 4.4. Implementation-wise, it is invoked via a single LLM call with a fixed system prompt and the role-token-annotated trace as the only data input. The auditor returns a structured JSON output with fields `is_malicious`, `confidence`, and `reason`. A sample is blocked only when `is_malicious=true` and `confidence` ≥ 0.5 . The LLM call uses `temperature=0.0`, `max_tokens=192`, JSON formatting, and up to three retries on malformed output. The auditor backend is treated as a configurable component. Table 5.8 lists the candidate LLM backends evaluated in this work, covering self-hosted models via Ollama and cloud-based proprietary models via OpenRouter. For accuracy-oriented experiments (Section 6.2), the default backend is `gpt-4.1-mini` via OpenRouter, which provides stable and high-quality judgments during detector selection. For latency-oriented experiments (Section 6.3), the default backend is the local `llama3.1:8b` model served by Ollama, since backend latency and queueing behavior must be measured in a controllable serving environment. For local models, the effective context length depends on runtime configuration rather than theoretical model limits.

Table 5.8: Candidate LLM Backends for Layer 3 Semantic Auditing

Deployment	Model ID	Name	Access
Self-hosted	llama3.1:8b	Llama 3.1 8B	Ollama
	mistralai/mistral-7b	Mistral 7B	Ollama
	google/gemma-2-9b	Gemma 2 9B	Ollama
Cloud-based	google/gemini-2.5-flash	Gemini 2.5 Flash	OpenRouter
	openai/gpt-5.4	GPT-5.4	OpenRouter
	openai/gpt-4o	GPT-4o	OpenRouter
	openai/gpt-4o-mini	GPT-4o-mini	OpenRouter

The three layers are wired in cascade as specified in Section 4.1: each layer's decision is computed before the next layer is invoked, and a malicious verdict at any layer terminates execution without invoking subsequent layers.

Chapter 6

Evaluation and Results

This chapter evaluates the proposed 3-layer filtering framework in terms of detection effectiveness, runtime efficiency, and practical attack prevention. The evaluation is conducted on the held-out test set constructed in Chapter 5, which contains 5,445 expanded interaction-history samples. Since each sample represents a progressively accumulated agent execution trace, the evaluation focuses on whether malicious behavior can be detected during the agent reasoning process rather than from isolated inputs only. The chapter follows three issues: Section 6.2 compares layer configurations, Section 6.3 analyzes the performance–latency trade-off, and Section 6.4 evaluates the actual attack success rate under different defenses.

6.1 Evaluation Overview

The evaluation uses both detection-oriented and system-oriented metrics. Precision measures how many samples flagged as malicious are truly malicious, while recall measures how many malicious samples are successfully detected. F1-score summarizes the balance between precision and recall and is used as the main detection-performance metric because the defense must avoid both missed attacks and excessive false alarms. Accuracy is reported as a general reference but is not used as the primary metric because the data distribution may be imbalanced. For system efficiency, latency per sample is measured as the end-to-end pipeline time from the moment a sample enters the defense system until the final decision is produced. In the backend-concurrency experiments, Layer 3 latency is measured as observed client-side latency from request submission to response reception; therefore, it includes both LLM serving time and backend queueing or contention delay. The Layer 2 and Layer 3 processing ratios are also reported because they explain how much traffic is forwarded to deeper layers and why reducing Layer 3 calls directly lowers total pipeline latency under concurrent backend load. Finally, Attack Success Rate (ASR) is used in Section 6.4 to measure whether attacks actually succeed

after passing the defense, making it the most direct security-impact metric.

The evaluated configurations are denoted by the ordered combination of layers defined in Chapter 4. For example, L1-L2-L3 denotes the full cascaded pipeline in which the interaction history is first checked by Layer 1, then Layer 2, and finally Layer 3 when escalation is required. Therefore, the evaluation compares not only detection performance across different layer combinations, but also how effectively each configuration reduces the number of samples requiring expensive LLM auditing.

6.2 Configuration Effectiveness of 3-Layer Filtering

This section answers the first issue: which layer configuration provides the best balance between detection effectiveness and runtime efficiency. Each configuration is named according to the actual execution order of the defense layers. For example, L1-L2-L3 means that the interaction history is first processed by Layer 1, then Layer 2, and finally Layer 3 when escalation is required. The configuration name therefore reflects both the subset of layers used and their execution order. For two-layer combinations, although six orderings are mathematically possible, only four are evaluated in Table 6.1. The remaining two are excluded because placing the heavyweight LLM auditor (L3) before lightweight filters (L1, L2) violates the cascaded pruning principle and offers no efficiency advantage. The complete mapping between layer settings and layer combinations is given in Table 6.1. The purpose of this comparison is to verify whether the three layers play complementary roles. If a single layer already achieved high precision, recall, F1-score, and low latency simultaneously, the cascaded design would not be necessary. However, the results in Table 6.1 and Figure 6.1 show that each individual layer has a clear bias. L1 achieves very high recall of 0.9915, but its precision is only 0.7911 and its FPR is 0.2577, meaning L1 is effective as a high-sensitivity anomaly detector but also flags many benign-but-unusual traces as suspicious. This behavior is expected, since the autoencoder is trained to model benign behavior and is intentionally tuned to avoid missing potential attacks. L2 shows the opposite behavior: it achieves very high precision of 0.9960 and a very low FPR of 0.0033, but its recall is only 0.8197, indicating that the supervised classifier is reliable when it recognizes known malicious patterns but misses a considerable portion of unseen, ambigu-

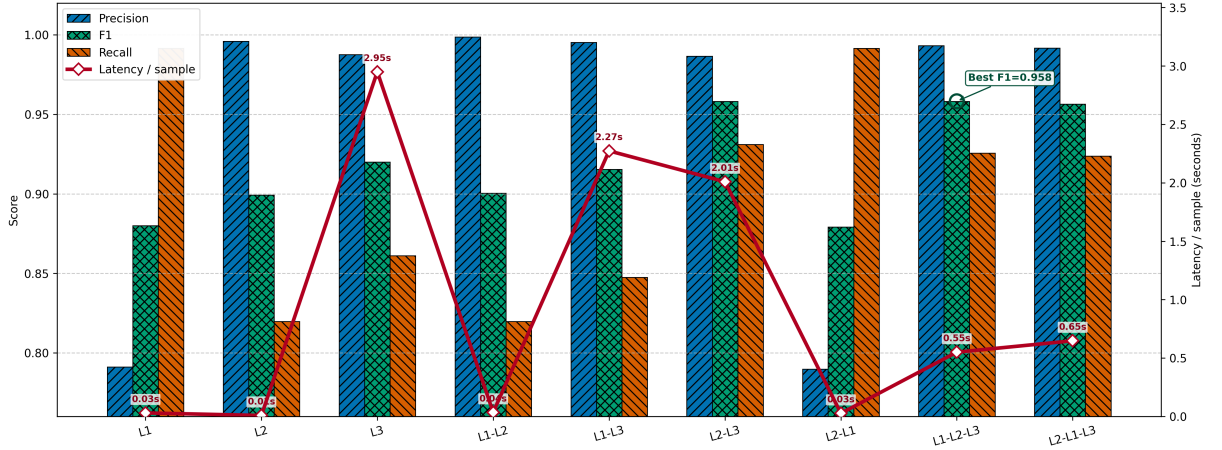


Figure 6.1: Layer Configuration Comparison

ous, or correlated attacks. L3 achieves stronger semantic reasoning than L1 and L2 alone, with precision of 0.9877 and F1-score of 0.9201, but its latency reaches 2.9504 seconds per sample because every sample is audited by the LLM. These single-layer results demonstrate that no individual layer simultaneously satisfies the requirements of high detection, low false alarm, and practical latency.

Table 6.1: Layer configuration comparison across evaluated configurations

Layer Setting	Configuration	Precision	Recall	F1	FPR	FNR	Accuracy	Latency / Sample (s)
1-Layer	L1	0.7911	0.9915	0.8801	0.2577	0.0085	0.8659	0.0300
	L2	0.9960	0.8197	0.8993	0.0033	0.1803	0.9089	0.0084
	L3	0.9877	0.8612	0.9201	0.0106	0.1388	0.9258	2.9504
2-Layer	L1-L2	0.9986	0.8197	0.9004	0.0011	0.1803	0.9100	0.0352
	L1-L3	0.9952	0.8475	0.9154	0.0040	0.1525	0.9223	2.2731
	L2-L3	0.9867	0.9311	0.9581	0.0124	0.0689	0.9596	2.0092
	L2-L1	0.7897	0.9915	0.8792	0.2598	0.0085	0.8648	0.0265
3-Layer	L1-L2-L3	0.9932	0.9256	0.9582	0.0062	0.0744	0.9600	0.5502
	L2-L1-L3	0.9917	0.9237	0.9565	0.0077	0.0763	0.9583	0.6477

The two-layer configurations further show why the final design requires all three layers. L1-L2 achieves very high precision of 0.9986 and the lowest FPR of 0.0011, but its recall remains 0.8197, essentially the same as L2 only. Because L1 has near-perfect recall, almost all attacks

are escalated to L2, so the overall recall is determined by L2’s recognition capability; unresolved attacks that L2 cannot confidently classify are forwarded by default in the absence of L3. L1-L3 improves semantic coverage compared with L1-L2, but its latency remains high at 2.2731 seconds per sample because too many samples are sent to Layer 3. L2-L3 reaches an F1-score of 0.9581, which is almost identical to the proposed L1-L2-L3 configuration, but its latency is 2.0092 seconds per sample. The reason is that L2-L3 sends 3,222 samples to Layer 3, while L1-L2-L3 sends only 1,168 samples. Therefore, L2-L3 is a strong detection baseline but is much less efficient because it does not use L1 as an early traffic regulator before LLM auditing.

The proposed L1-L2-L3 configuration provides the strongest overall trade-off. It achieves precision of 0.9932, recall of 0.9256, F1-score of 0.9582, FPR of 0.0062, and latency of 0.5502 seconds per sample. Compared with L3 alone, it reduces latency from 2.9504 to 0.5502 seconds per sample because only a subset of unresolved cases reaches the LLM auditor. Compared with L2 alone, it improves recall from 0.8197 to 0.9256 because Layer 3 can recover attacks missed by the supervised classifier. Compared with L1 alone, it improves precision from 0.7911 to 0.9932 because L2 and L3 remove false alarms generated by the anomaly detector. The alternative three-layer order, L2-L1-L3, achieves similar performance with F1-score of 0.9565, confirming that the two lightweight layers are complementary; however, L1-L2-L3 has lower latency because it allows the anomaly detector to act first as a low-cost front gate. Overall, the results support the main architectural claim: the three-layer framework is effective not because it simply stacks more detectors, but because each layer compensates for the weaknesses of the others. L1 provides broad anomaly coverage, L2 provides high-precision recognition of known attacks, and L3 provides semantic reasoning for ambiguous and correlated cases.

As an additional analysis of Layer 3, the final configuration-related experiment compares different LLM backends for the semantic auditor while keeping the routing configuration fixed at L1-L2-L3. This experiment separates the effect of the cascaded architecture from the effect of the chosen LLM backend. As shown in Figure 6.2 and Table 6.2, GPT-5.4 achieves the best F1-score of 0.9691 with the lowest FPR of 0.0015 and latency of 0.614 seconds per sample. Gemini 2.5 Flash is also strong, with F1-score of 0.9662 and recall of 0.9522. GPT-4.1-mini achieves high precision of 0.9960 and F1-score of 0.9567, making it a reasonable backend for

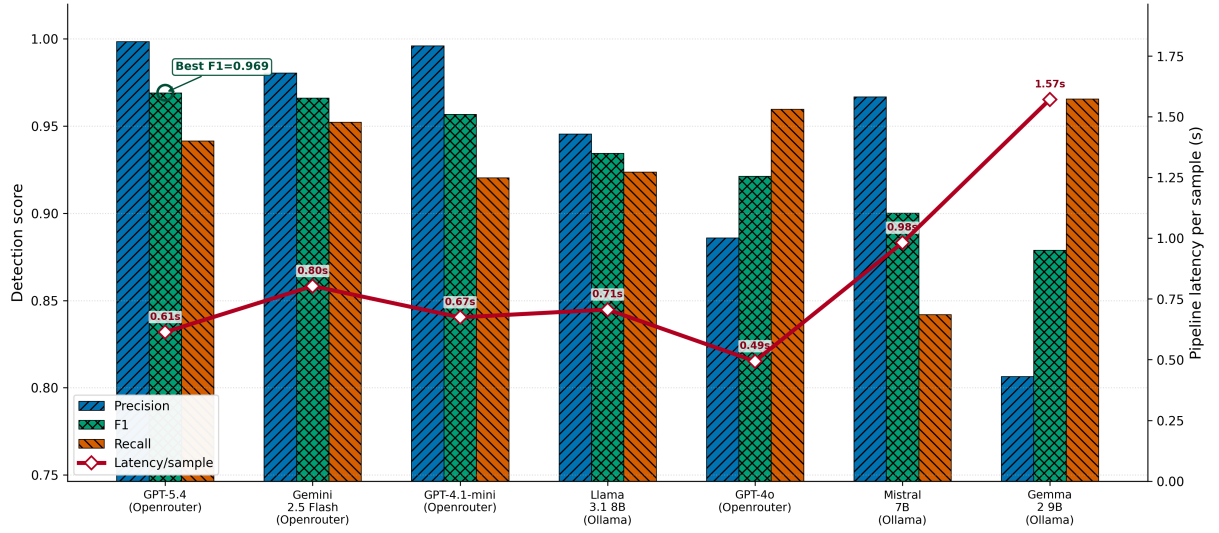


Figure 6.2: Layer 3 Backend Comparison

configuration testing. Among local backends, Llama 3.1 8B achieves F1-score of 0.9345 and recall of 0.9237 with latency of 0.706 seconds per sample, making it a practical local backend for controlled latency and queueing experiments. Other backends show less favorable trade-offs: Gemma 2 9B reaches the highest recall of 0.9656 but suffers from weak precision and an FPR of 0.2281, while GPT-4o achieves the lowest latency at 0.494 seconds per sample but its FPR of 0.1217 is too high for this setup. These results indicate that backend selection affects the accuracy–latency balance of Layer 3, and that high recall alone is not sufficient when it comes at the cost of substantially increased false positives. Nevertheless, the architectural conclusion remains unchanged: the cascaded framework is necessary because it limits the number of samples that depend on expensive LLM auditing.

Table 6.2: Layer 3 backend comparison under Mode 11

Model	Backend	Latency / Sample (s)	Precision	Recall	F1	FPR	FNR
GPT-5.4	OpenRouter	0.614	0.9984	0.9415	0.9691	0.0015	0.0585
Gemini 2.5 Flash	OpenRouter	0.803	0.9806	0.9522	0.9662	0.0186	0.0478
GPT-4.1-mini	OpenRouter	0.674	0.9960	0.9204	0.9567	0.0036	0.0796
Llama 3.1 8B	Ollama	0.706	0.9454	0.9237	0.9345	0.0525	0.0763
GPT-4o	OpenRouter	0.494	0.8859	0.9596	0.9213	0.1217	0.0404
Mistral 7B	Ollama	0.981	0.9668	0.8419	0.9001	0.0284	0.1581
Gemma 2 9B	Ollama	1.571	0.8064	0.9656	0.8789	0.2281	0.0344

6.3 Performance–Latency Trade-off

This section answers the second issue: how the system balances detection performance and runtime cost. Since Layer 3 is the most expensive component, the main efficiency question is how many samples are escalated to the LLM auditor and how backend load affects the observed end-to-end latency. This section is organized into three analyses. First, the backend latency of Layer 3 is measured under different client-side concurrency levels to quantify the cost of LLM-based auditing. Second, the Layer 1 threshold is varied under a fixed concurrent load to examine how threshold selection regulates downstream traffic. Third, the proportion of benign traffic is varied to evaluate how the system behaves under realistic, benign-heavy deployment conditions. The measured latency is the total pipeline latency per sample, including Layer 1 processing, Layer 2 processing, Layer 3 calls, and observed backend contention.

Figure 6.3 shows the observed Layer 3 call latency when the number of concurrent agent requests increases from 1 to 4. The serving time represents the baseline LLM processing time, while the queueing/contention time represents the additional delay caused by multiple requests competing for the same backend capacity. When concurrency increases, the total observed Layer 3 latency grows from 3.39 seconds at concurrency 1 to 11.99 seconds at concurrency 4. This result should be interpreted as a deployment-dependent reference rather than a universal constant,

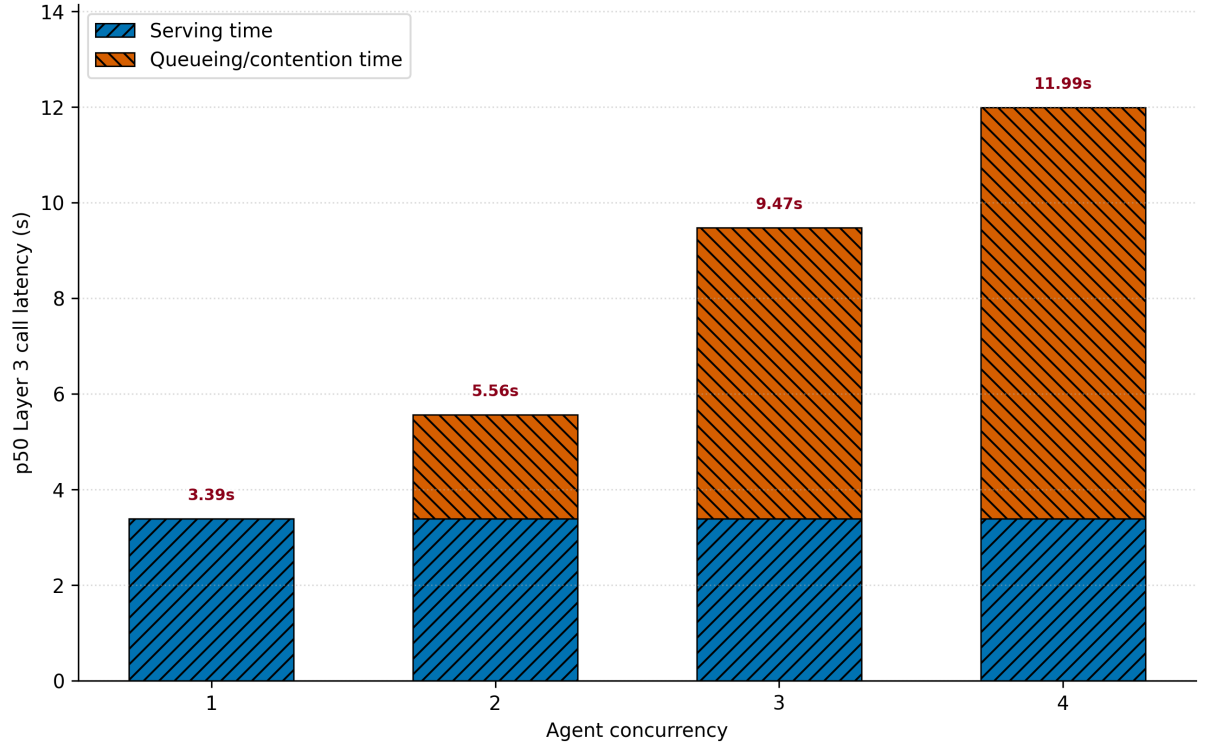


Figure 6.3: Layer 3 Serving and Queueing Latency under Concurrent Requests

because the absolute latency depends on the server hardware, backend scheduler, model backend, and number of simultaneous requests. However, it clearly shows the system-level trend: Layer 3 is expensive not only because LLM inference itself is slow, but also because backend contention can significantly increase the observed latency under concurrent serving. Therefore, reducing the number of samples escalated to Layer 3 is necessary for practical deployment.

Based on this observation, the following threshold and distribution experiments use Layer 3 client-side concurrency set to 4. This setting represents the heaviest backend-load condition in the concurrency test and is used to evaluate whether the cascaded design remains effective when multiple LLM auditing requests compete for backend capacity. The results in Figure 6.4 show that increasing the Layer 1 threshold consistently reduces downstream workload. This experiment is conducted under the L1-L2-L3 configuration. When the threshold increases from 0.05 to 0.25, the Layer 2 processing ratio decreases from 62.17% to 45.71%, and the Layer 3 processing ratio decreases from 21.45% to 5.82%. As a result, total latency per sample decreases from 2.80 seconds to 0.81 seconds. This confirms that the Layer 1 threshold is not only a model hyperparameter, but also a system-level control knob that regulates how much traffic reaches

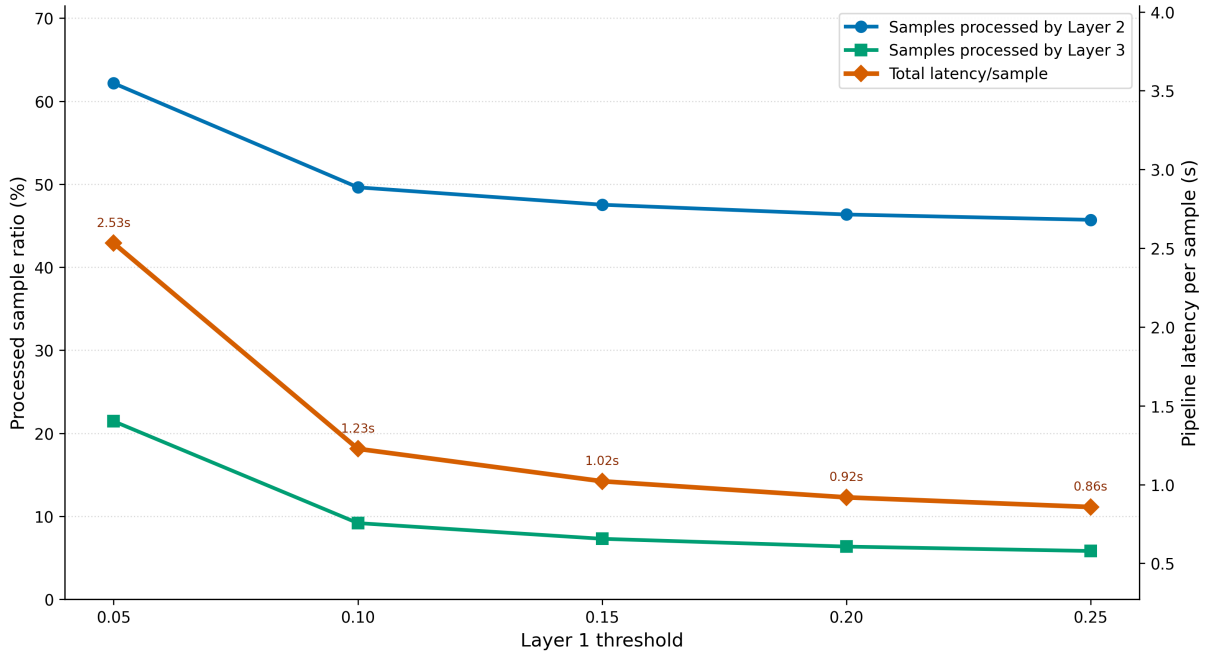


Figure 6.4: Threshold Load and Latency

the expensive LLM backend.

The corresponding detection performance is shown in Figure 6.5. This experiment is conducted under the L1-L2-L3 configuration. As the threshold increases, precision improves from 0.9454 to 0.9987 because fewer benign but unusual samples are escalated and misclassified as malicious. However, recall gradually decreases from 0.9226 to 0.8856 because a higher threshold allows more suspicious samples to pass Layer 1 without deeper inspection. F1-score reaches its best value of 0.9481 at threshold 0.10, where the system maintains high precision while preserving sufficient recall. Threshold 0.15 provides a more latency-oriented operating point, reducing latency while maintaining F1-score of 0.9461. Thresholds 0.20 and 0.25 further reduce latency, but recall falls below 0.90, indicating that the system becomes too permissive for security-sensitive deployment. The choice of threshold should therefore be guided by the target domain: a high-security setting should prefer lower thresholds to preserve recall, while a latency-sensitive setting may accept a slightly higher threshold as long as the F1-score remains stable.

The third analysis evaluates high-benign workload distributions, which represent a more realistic deployment setting because most production traffic is expected to be benign. The benign ratio is varied from 90% to 98% with the Layer 1 threshold fixed at 0.25. This threshold

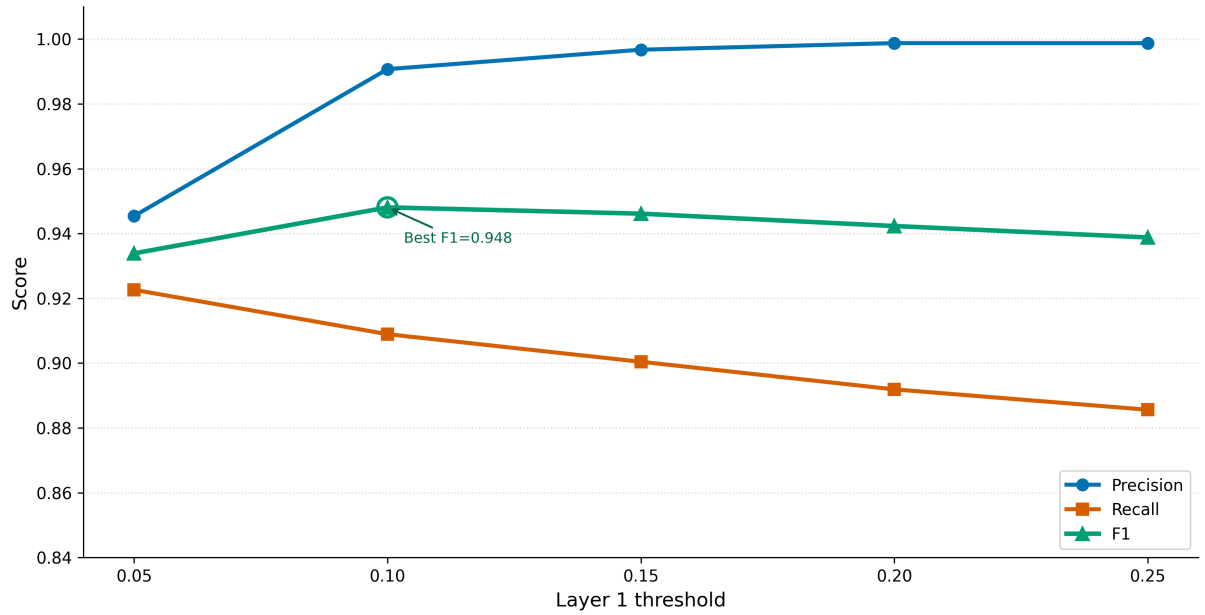


Figure 6.5: Threshold Performance Stability

is selected as a latency-oriented operating point from the previous threshold sweep because it minimizes the Layer 3 processing ratio and total latency under backend pressure. Therefore, this experiment does not claim that 0.25 is universally optimal; instead, it evaluates how the cascaded system behaves under a practical low-latency configuration when the workload becomes increasingly benign-heavy. Figure 6.6 shows that as the benign ratio increases, fewer samples are escalated to L2 and L3. The Layer 2 processing ratio decreases from 9.09% at 90% benign traffic to 2.16% at 98% benign traffic, and the Layer 3 processing ratio decreases from 1.98% to 0.65%. This is because L1 and L2 already filter most of the clearly benign or clearly malicious samples, leaving only a small fraction of ambiguous cases for L3. As a result, latency decreases from 0.40 seconds per sample at 90% benign traffic to 0.15 seconds per sample at 98% benign traffic. This result supports the practical value of the cascaded design: the system becomes cheaper when deployed in benign-heavy environments, which is exactly the expected condition for real-world enterprise agents.

The detection performance under high-benign workload is shown in Figure 6.7. F1-score remains within a moderate range from 0.8544 to 0.8961, with the best value of 0.8961 at 94% benign traffic. Recall stays around 0.79 to 0.83, showing that the system still detects a large portion of malicious samples even when attacks become rare. Precision decreases from 0.9912 at 90% benign traffic to 0.9362 at 98% benign traffic. This decrease should not be interpreted

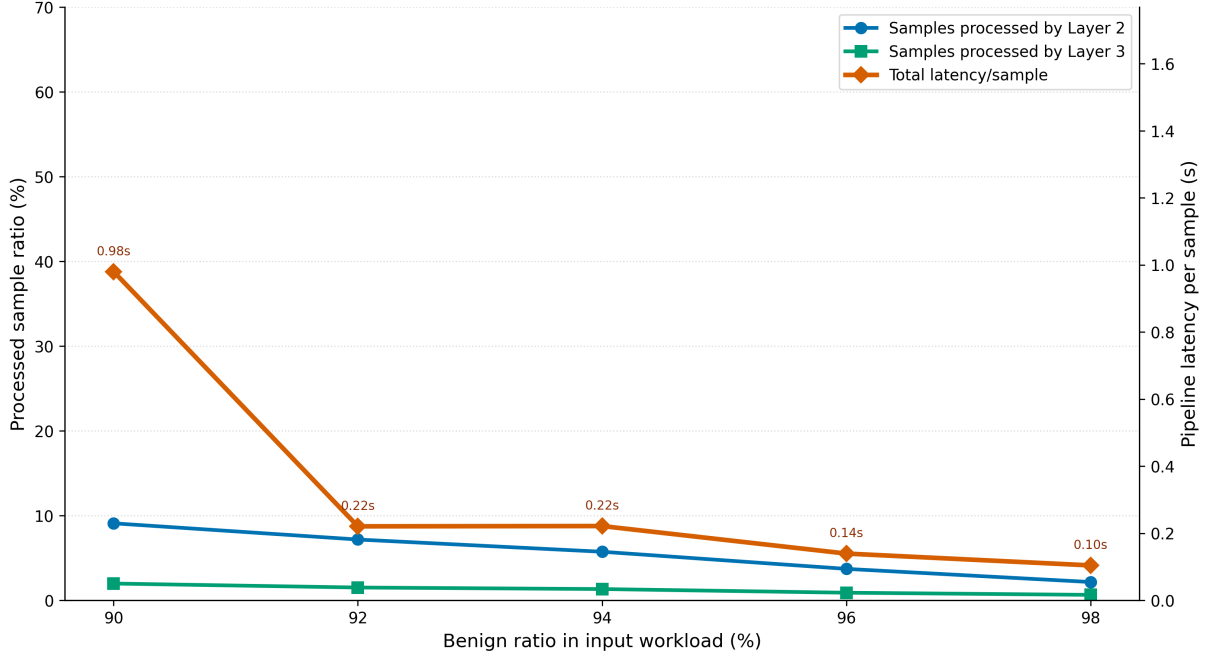


Figure 6.6: High-Benign Load and Latency

as a large increase in false positives; it is mainly a base-rate effect, because when malicious samples are rare, even a very small number of false positives has a disproportionate impact on precision. To verify this interpretation, the 98% benign experiment reports an FPR of 0.0011, indicating that the absolute false-positive rate remains very low even when precision becomes more sensitive to the small number of malicious samples. Overall, the high-benign experiment shows that the proposed defense maintains stable behavior under realistic class imbalance while significantly reducing latency through progressive traffic pruning.

6.4 Impact of Attacks

This section answers the third issue: whether the proposed defense reduces the actual success rate of input injection attacks in an LLM-based agent. Unlike the previous sections, this section uses Attack Success Rate (ASR) as the main metric because ASR directly measures whether an attack still succeeds after passing the defense. The experiment compares no defense, Sentinel [5], RAGuard [21], ToolResultParsing [33], the combined baseline, and the proposed 3-layer defense across prompt injection, RAG poisoning, tool injection, and correlated multi-channel attacks.

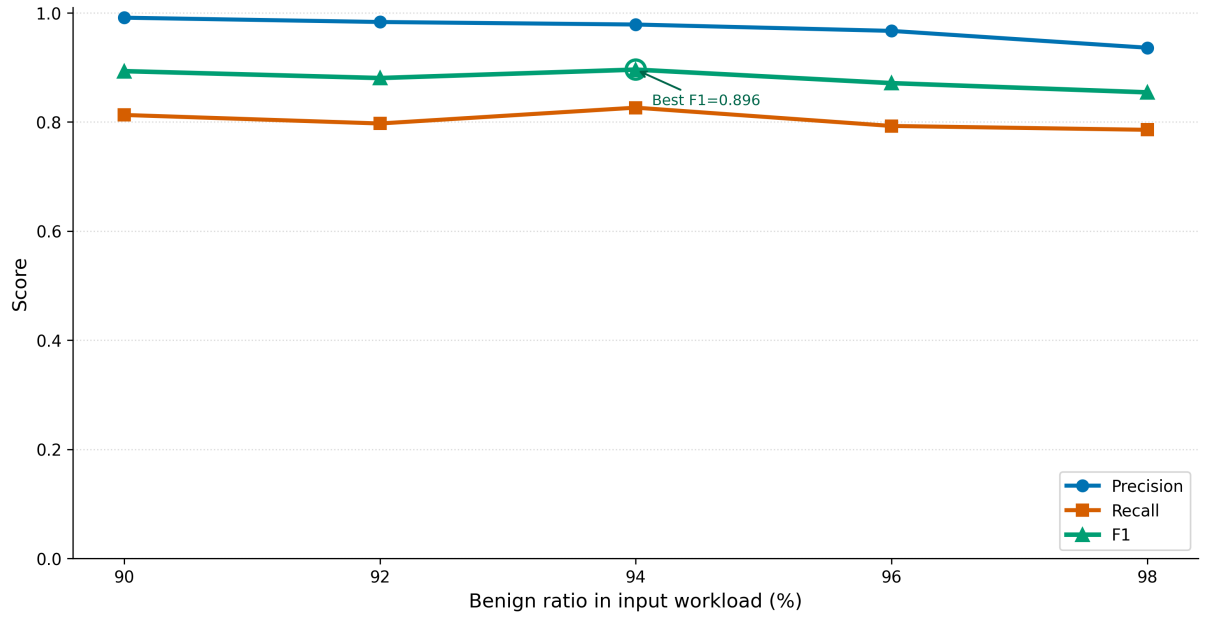


Figure 6.7: High-Benign Performance Stability

Figure 6.8 shows that the no-defense setting has the highest mean ASR of 0.4999. Direct prompt injection has a low ASR of 0.0500, while RAG poisoning, tool injection, and correlated attacks reach much higher ASR values of 0.5505, 0.9267, and 0.4725, respectively. Channel-specific defenses reduce ASR mainly within their protected surfaces. Sentinel [5] reduces prompt injection ASR to 0.0000, but still leaves high ASR for RAG poisoning, tool injection, and correlated attacks. RAGuard [21] reduces RAG ASR from 0.5505 to 0.3578, while ToolResultParsing [33] reduces tool ASR from 0.9267 to 0.2533. The combined baseline, which integrates Sentinel [5], RAGuard [21], and ToolResultParsing [33], further reduces mean ASR to 0.1574, but still leaves non-zero ASR for RAG poisoning, tool injection, and correlated attacks.

The proposed 3-layer defense achieves ASR of 0.0000 across all four attack categories. This result shows that combining channel-specific defenses improves protection, but does not eliminate cross-channel attacks. The key insight is that LLM-agent security should be evaluated at the full-interaction level rather than by isolated channels only. Since malicious intent can emerge after prompt, retrieval, tool outputs, and reasoning steps are aggregated, full interaction-history auditing is necessary to prevent actual agent compromise.

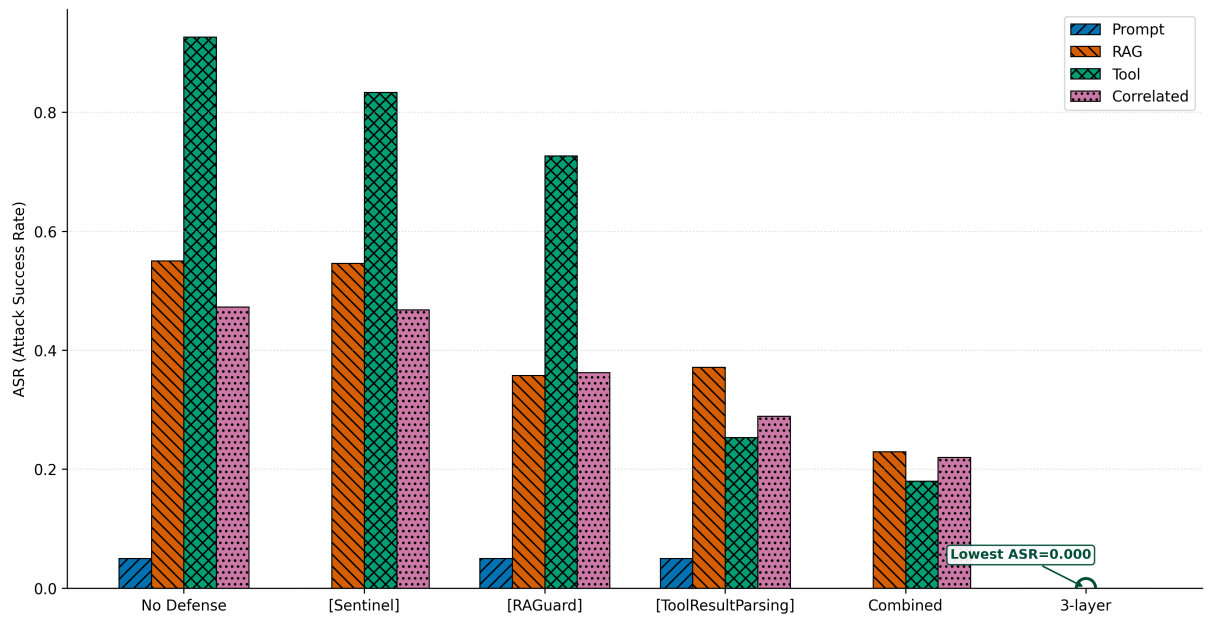


Figure 6.8: ASR Comparison Across Defenses

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis studied the problem of detecting and preventing input injection attacks in LLM-based agents. As discussed in earlier chapters, LLM-based agents are no longer passive text generation systems. They can retrieve external knowledge, call tools, and make multi-step decisions through accumulated interaction histories. While these capabilities enable more autonomous and practical applications, they also expand the attack surface. Malicious instructions may be injected not only through user prompts, but also through retrieved documents, tool outputs, and multi-step reasoning contexts. This makes input injection an interaction-level security problem rather than a purely input-level problem.

The core challenge addressed in this thesis is that malicious intent may not be visible in any single input. In correlated multi-channel attacks, each individual component can appear benign, but the agent may form a malicious instruction after aggregating information from prompts, RAG memory, tool outputs, and previous reasoning steps. Therefore, defenses that inspect only isolated prompts, retrieved documents, or individual tool decisions are insufficient. This thesis addressed this gap by proposing a full-interaction-level defense paradigm that audits the aggregated interaction history instead of treating each input channel independently.

To implement this paradigm, this thesis proposed a cascaded 3-layer filtering framework. Layer 1 applies embedding-based anomaly detection to capture abnormal interaction patterns, especially potential unseen or zero-day attacks. Layer 2 applies supervised transformer classification to detect known injection patterns and reduce false positives introduced by the anomaly detector. Layer 3 applies LLM-based semantic auditing to verify ambiguous and complex cases using the full interaction history. This design allows lightweight layers to process most traffic, while reserving expensive LLM-based reasoning for only a small fraction of suspicious cases.

The evaluation results in Chapter 6 answer the three main issues of this thesis. For the first issue, the results show that no single layer can achieve the desired balance among recall, precision, false positive rate, and latency. Layer 1 provides broad anomaly coverage but produces more false positives. Layer 2 achieves high precision but misses some unseen or ambiguous attacks. Layer 3 provides stronger semantic reasoning but is too costly when applied to all samples. The full 3-layer configuration achieves the best overall trade-off because the three layers play complementary roles. This confirms that the proposed architecture is effective not because it simply stacks more detectors, but because each layer compensates for the limitations of the others.

For the second issue, the results show that the Layer 1 threshold is a key mechanism for controlling the performance–latency trade-off. A lower threshold increases detection sensitivity but sends more samples to deeper layers, especially Layer 3, resulting in higher latency. A higher threshold reduces Layer 3 traffic and improves efficiency, but may reduce recall if suspicious traces are allowed to pass too early. The experiments demonstrate that the cascaded design can significantly reduce the number of samples requiring LLM-based auditing while maintaining strong detection performance. This shows that the proposed framework is not only a detection system, but also a traffic regulation mechanism for making interaction-level auditing practical.

For the third issue, the ASR evaluation demonstrates the practical security benefit of the proposed defense. Without defense, indirect attacks through RAG and tool channels achieve high attack success rates, showing that multi-channel attacks are more dangerous than direct prompt injection in LLM-based agents. Channel-specific defenses reduce attack success within their protected surfaces, but they still leave non-zero ASR when attacks are distributed across channels. Even the combined baseline remains vulnerable because it still lacks a unified view of the full reasoning trajectory. In contrast, the proposed 3-layer defense reduces ASR to zero across the evaluated attack categories, confirming the necessity of full interaction-history auditing for correlated multi-channel attacks.

Overall, this thesis concludes that securing LLM-based agents requires moving beyond isolated input filtering and local execution checking. Since malicious behavior can emerge only after multiple benign-looking inputs are accumulated, the defense must evaluate whether the

agent’s overall interaction history remains aligned with the user’s original intent. The proposed 3-layer filtering framework provides a practical realization of this idea by combining anomaly detection, supervised classification, and semantic auditing. The results show that this approach can improve security against input injection attacks while maintaining practical efficiency, making full-interaction-level defense a promising direction for real-world LLM-agent protection.

7.2 Future Work

Although the proposed framework demonstrates strong effectiveness against prompt injection, RAG poisoning, tool injection, and correlated multi-channel attacks, several directions remain for future work. First, this thesis mainly focuses on text-based interaction histories, including prompts, retrieved documents, tool outputs, and reasoning traces. However, future LLM-based agents are increasingly expected to process multimodal information such as images, audio, video, screenshots, and structured interface states. These modalities may introduce new forms of input injection, where malicious instructions are hidden in visual or multimodal content rather than plain text. Therefore, future work should extend the interaction-history representation from text-only traces to multimodal traces, allowing the defense system to audit how textual and non-textual inputs jointly influence agent reasoning.

Second, the current evaluation is conducted on a LangGraph-based testbed with controlled attack scenarios. This design is useful because it provides reproducible traces and allows direct measurement of attack success. However, real-world agent systems may differ in orchestration framework, tool design, memory implementation, and user workflow. Future work should evaluate the proposed framework across additional agent frameworks, such as LangChain agents, AutoGen, CrewAI, and enterprise-oriented agent platforms. Such evaluation would help verify whether the full-interaction defense paradigm generalizes across different agent architectures and deployment environments.

Third, future work should further optimize the deployment efficiency of the proposed cascade. Although the current design already reduces the number of samples reaching Layer 3, real-world systems may require stricter latency guarantees under high concurrency and changing workloads. Future research can explore adaptive threshold selection, dynamic routing policies,

lightweight semantic auditors, and model compression techniques. For example, the system may automatically adjust the Layer 1 threshold based on workload, backend congestion, or domain-specific risk level. This would allow the defense to maintain a better balance between security and latency under practical runtime conditions.

Finally, future work can extend the evaluation from detection and prevention metrics to broader operational metrics. In real deployments, a defense system must not only block attacks, but also preserve benign task completion, minimize unnecessary interruptions, and provide understandable explanations to users or administrators. Therefore, future studies should evaluate task success rate, explanation quality, recovery behavior after blocking, and integration with monitoring dashboards. These directions would make the proposed framework more suitable for practical enterprise LLM-agent systems.



References

- [1] J. Lee, “ReAct vs Plan-and-Execute: A Practical Comparison of LLM Agent Patterns,” <https://dev.to/jamesli/react-vs-plan-and-execute-a-practical-comparison-of-llm-agent-patterns-4gh9>, Nov. 2024, DEV Community.
- [2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624>
- [3] Lakera, “Indirect Prompt Injection: The Hidden Threat Breaking Modern AI Systems,” <https://www.lakera.ai/blog/indirect-prompt-injection>, 2025, Lakera AI.
- [4] Z. Wang *et al.*, “MindGuard: Tracking, Detecting, and Attributing MCP Tool Poisoning Attack via Decision Dependence Graph,” Aug. 2025. [Online]. Available: <https://arxiv.org/abs/2508.20412>
- [5] D. Ivry and O. Nahum, “Sentinel: SOTA Model to Protect Against Prompt Injections,” Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.05446>
- [6] Aim Labs, “EchoLeak: Zero-click Microsoft 365 Copilot Data Leak,” <https://www.aim.security/lp/aim-labs-echoleak-blogpost>, Jun. 2025, Aim Security Blog.
- [7] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models,” in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025. [Online]. Available: <https://arxiv.org/abs/2402.07867>

- [8] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, “Prompt Injection Attack to Tool Selection in LLM Agents,” Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.19793>
- [9] ProtectAI, “Fine-Tuned DeBERTa-v3 for Prompt Injection Detection,” <https://huggingface.co/ProtectAI/deberta-v3-base-prompt-injection>, 2023, Hugging Face Model Hub.
- [10] J. Tint, “PROOD: A Simple LLM Out-of-Distribution Guardrail Leveraging Response Semantics,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 23 428–23 438.
- [11] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, “DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents,” Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.12104>
- [12] T. Zhang *et al.*, “AgentSentry: Mitigating Indirect Prompt Injection in LLM Agents via Temporal Causal Diagnostics and Context Purification,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.22724>
- [13] H. An *et al.*, “IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents,” Aug. 2025. [Online]. Available: <https://arxiv.org/abs/2508.15310>
- [14] S. Chennabasappa *et al.*, “LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents,” May 2025. [Online]. Available: <https://arxiv.org/abs/2505.03574>
- [15] B. Zhang *et al.*, “Practical Poisoning Attacks against Retrieval-Augmented Generation,” Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2504.03957>
- [16] ProtectAI, “Fine-Tuned DeBERTa-v3-base for Prompt Injection Detection (v2),” <https://huggingface.co/ProtectAI/deberta-v3-base-prompt-injection-v2>, 2024, Hugging Face Model Hub.

- [17] Y. Shao *et al.*, “POISONCRAFT: Practical Poisoning of Retrieval-Augmented Generation for Large Language Models,” May 2025. [Online]. Available: <https://arxiv.org/abs/2505.06579>
- [18] N. Kumar, “AI Bastion: Fine-Tuned DeBERTa-v3 for Prompt Injection & Jailbreak Detection,” <https://huggingface.co/neeraj-kumar-47/aibastion-prompt-injection-jailbreak-detector>, 2025, Hugging Face Model Hub.
- [19] Y. Wang, S. Chen, R. Alkhudair, B. Alomair, and D. Wagner, “Defending Against Prompt Injection with DataFilter,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.19207>
- [20] K. Edemacu, V. M. Shashidhar, M. Tuape, D. Abudu, B. Jang, and J. W. Kim, “FilterRAG: Extracting Statistical Properties to Defend Against Knowledge Poisoning Attacks,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.02835>
- [21] Z. Cheng *et al.*, “Secure Retrieval-Augmented Generation against Poisoning Attacks,” Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.25025>
- [22] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably Robust RAG against Retrieval Corruption,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.15556>
- [23] N. Li, K. Zhang, K. Polley, and J. Ma, “Security considerations for artificial intelligence agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.12230>
- [24] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa, “Miniscope: A least privilege framework for authorizing tool calling agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.11147>
- [25] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, “Taming various privilege escalation in llm-based agent systems: A mandatory access control framework,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.11893>
- [26] D. Cheng and W.-K. Tsao, “Agent privilege separation in openclaw: A structural defense against prompt injection,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.13424>

- [27] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “Struq: Defending against prompt injection with structured queries,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.06363>
- [28] E. S. Mathew, “Enhancing security in large language models: A comprehensive review of prompt injection attacks and defenses,” *Journal on Artificial Intelligence*, vol. 7, no. 1, pp. 347–363, 2025. [Online]. Available: <http://www.techscience.com/jai/v7n1/64040>
- [29] A. Alzahrani, “Promptguard: A structured framework for injection-resilient language models,” *Scientific Reports*, vol. 16, no. 1, p. 1277, 2026.
- [30] deepset, “Prompt injections dataset,” <https://huggingface.co/datasets/deepset/prompt-injections>, 2023, accessed: 2026.
- [31] A. Chawla and A. Pachaar, “Implementing Planning Agentic Pattern From Scratch,” <https://www.dailydoseofds.com/ai-agents-crash-course-part-11-with-implementation/>, Apr. 2025, Daily Dose of Data Science.
- [32] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, “Natural Questions: A Benchmark for Question Answering Research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: <https://aclanthology.org/Q19-1026>
- [33] Q. Yu, X. Cheng, and C. Liu, “Defense Against Indirect Prompt Injection via Tool Result Parsing,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.04795>